

Formation FPGA

TP5 – Domaines d'horloge

1) Pour piloter nos deux LEDs, nous allons avoir besoin tout d'abord de 2 modules *LED_driver_bis*, tel que décrit dans la deuxième partie du TP4. Nous avons légèrement modifié ce module, en changeant comment le signal de sortie *end_cycle* est généré : ce signal est maintenant activé lorsque le clignotement fini, et non plus à la fin de la période off. Cela permettra d'éviter ce phénomène observé au TP précédent où la LED s'allume pendant un coup d'horloge avec l'ancienne couleur, le temps que le signal *end_cycle* ne produise la mise à jour du code couleur en entrée du module.

Le premier module aura ses sorties reliées au canaux RGB de la LED 0, et le second à ceux de la LED1.

Nous devons ensuite envisager le pilotage des entrées de nos modules *LED_driver*. En plus des signaux d'horloge et de reset asynchrone, que nous pouvons simplement passer en entrées de notre nouvelle architecture, nous avons aussi besoin de créer les autres signaux en entrées des modules *LED_driver*, *color_code* et *update*. Puisque nos deux LEDs doivent avoir le même comportement, un seul signal suffira pour piloter chaque paire d'entrées identiques pour les deux modules.

Pour ce qui est du signal *color_code*, on va le créer avec une FSM *Color* qui va faire un cycle entre trois états : « red », « blue » et « green », chacun produisant le code couleur associé (voir énoncé du TP4, partie 1, question 8). Cette FSM a en entrées les signaux d'horloges et de reset asynchrones globaux, ainsi qu'un signal *change_state* déclenchant le changement d'état (et donc de couleur) de la FSM.

D'après la description de l'énoncé, ce signal devrait être activé pendant un unique coup d'horloge après 10 clignotements d'une LED de la même couleur.

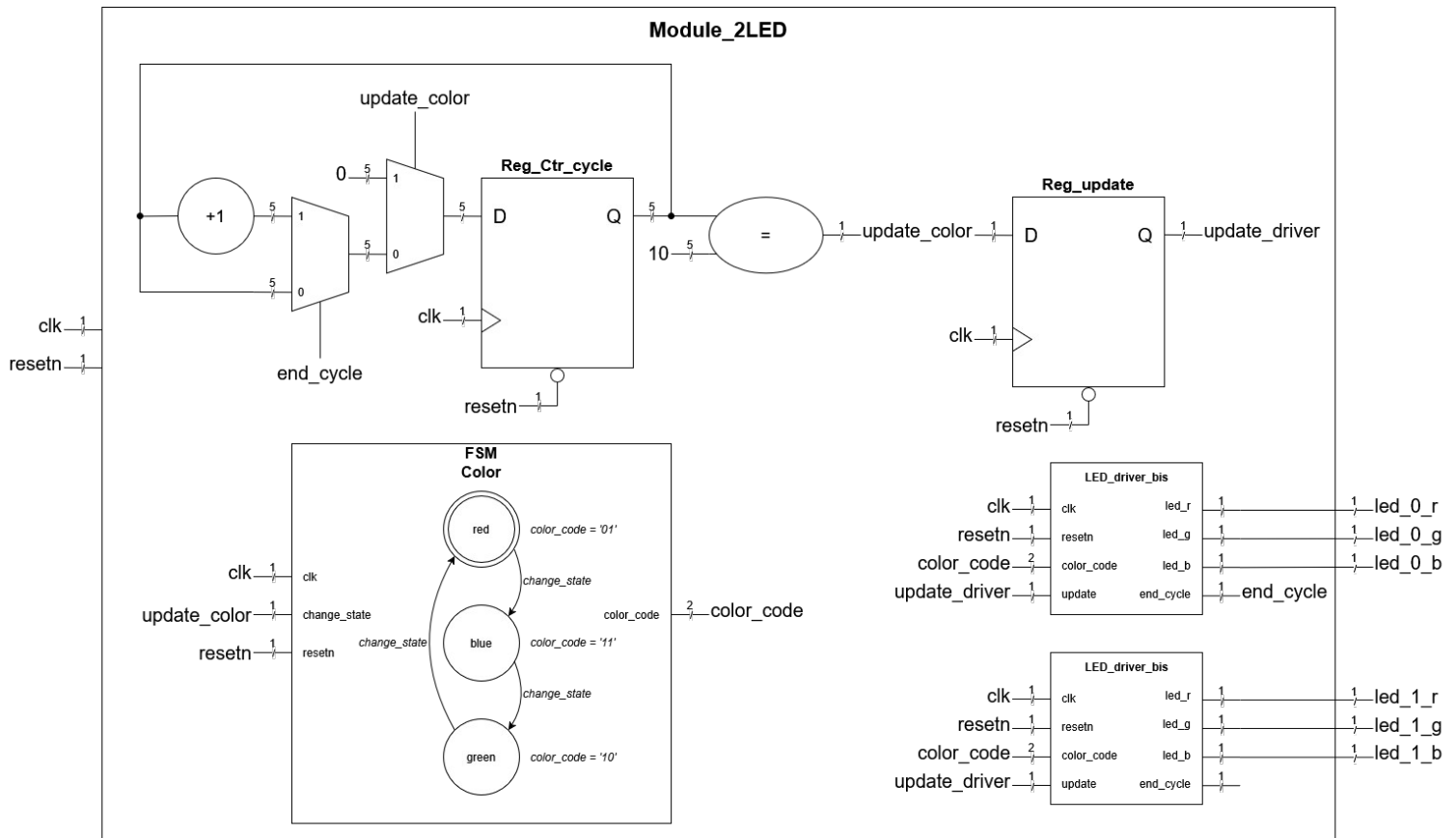
Pour créer ce signal, nous allons instaurer un compteur de cycle. Ce compteur sera stocké sur un registre de 4 bits (pour pouvoir compter jusqu'à 10). Le compteur s'incrémente à chaque fin de cycle, indiquée par le signal *end_cycle* en sortie d'un des 2 modules *LED_driver*. Lorsque le compteur atteint 10, on active un signal *update_color*, et on réinitialise le compteur à 0.

Ce signal *update_color* est donc relié à l'entrée *change_state* de la FSM *Color*.

Le signal *update_color* ne peut pas être utilisé en l'état pour déclencher aussi l'*update* des modules *LED_driver*, car la mise à jour du *color_code* en sortie de la FSM *Color* se fait à la fin du cycle d'horloge après le déclenchement de du changement d'état. Au moment où *update_color* devient haut, c'est encore l'ancienne valeur de *color_code* qui est en entrée des modules *LED_driver*.

Pour décaler d'un coup d'horloge le signal d'*update*, on va utiliser un registre qui prend en entrée le signal *update_color*, et qui le restitue un coup d'horloge plus tard dans le signal *update_driver*, relié aux entrées *update* des 2 modules *LED_driver*.

Voici l'architecture RTL correspondant à ce nouveau module *Module_2LED*, regroupant tous les éléments qu'on vient de lister :



2) Le code VHDL correspondant à cette architecture RTL a été traduit dans le fichier VHDL *Module_2LED.vhd*. Nous joignons aussi dans le dossier de rendu le code mis-à-jour pour le module *LED_driver_bis* (fichier *LED_driver_bis.vhd*), avec la modification sur la sortie *end_cycle* décrit à la question précédente.

En plus d'implémenter 2 modules *LED_driver_bis*, notre code décrit :

- une partie séquentielle, avec la gestion des registres compteur de cycle et update, ainsi que la FSM (changement d'état et effet de chaque état) ;
- une partie combinatoire, avec le calcul de la prochaine valeur du compteur de cycle, et le test sur la fin du compteur de cycle.

3) Notre module est testé dans un testbench *tb_Module_2LED.vhd*

On repart des derniers paramètres de simulation utilisés pour les testbenchs du TP4 :

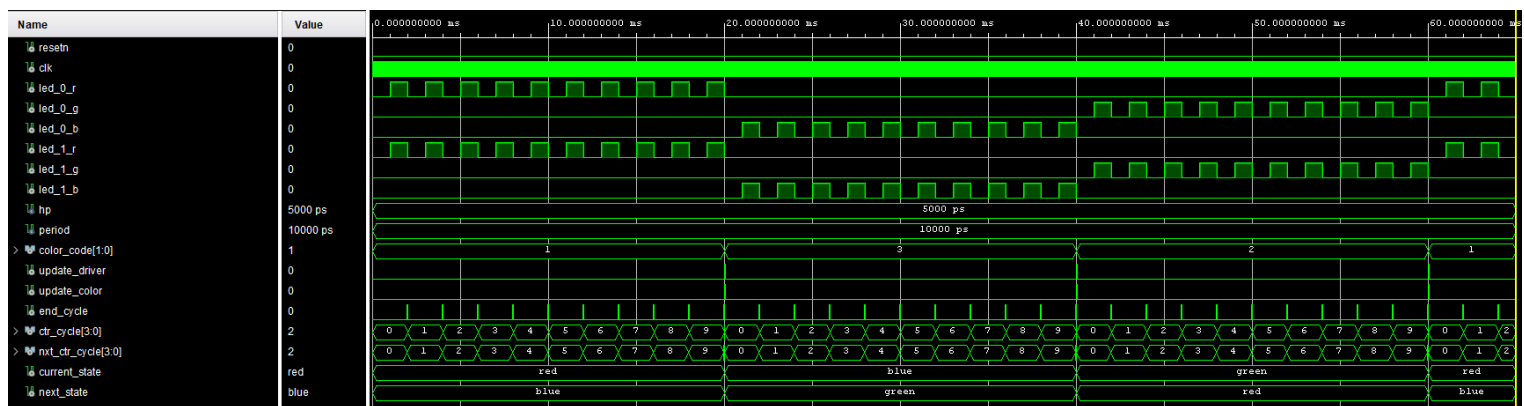
- Une horloge de fréquence 100 MHz, soit une période de 10 ns, avec un signal carré (demi-période 5 ns)
- Une *cible* du compteur de 100000, ce qui correspond à des cycles de 1 ms

Pendant notre simulation, on va observer principalement nos six sorties qui pilotent les LEDs (*led_0_r*, *led_0_g* et *led_0_b* pour la LED 0, *led_1_r*, *led_1_g* et *led_1_b* pour la LED 1).

On va aussi observer quelques signaux internes pour vérifier le bon fonctionnement :

- Le signal *color_code*, en sortie de la FSM *Color* et en entrée des modules *LED_driver_bis*.
- Les signaux *update_color* et *update_driver*, qui respectivement déclenchent le changement d'état de la FSM *Color* et l'*update* des modules *LED_driver_bis*.
- Les signaux sur 4 bits *ctr_cycle* et *nxt_ctr_cycle* du compteur de cycles.
- Le signal *end_cycle*, en sortie d'un des modules *LED_driver_bis*, qui déclenche l'incréméntation du compteur de cycle.
- Les signaux *current_state* et *next_state* qui contiennent les états (actuels et futurs) de la FSM *Color*.

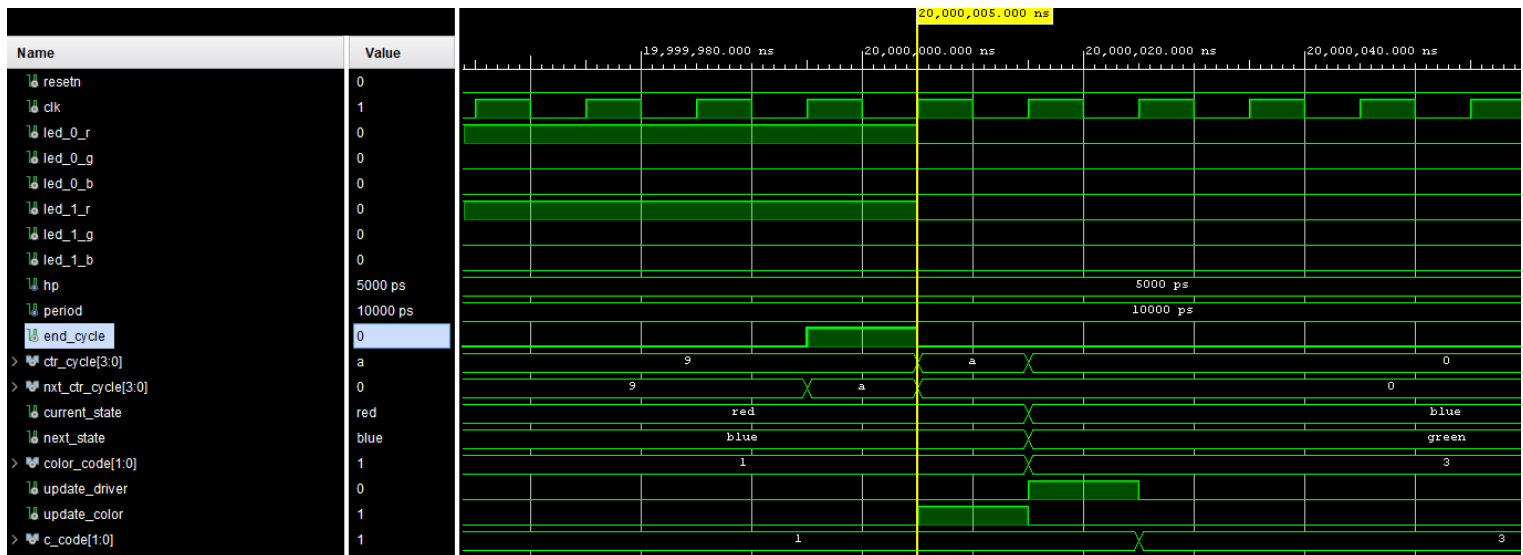
Voici le chronogramme observé pour 65 ms de simulation, sans intervention particulière sur les entrées du module :



De manière générale, on observe que le mécanisme de clignotement du module *LED_driver_bis*, introduit au TP précédent, produit toujours ce clignotement régulier.

On observe bien 10 clignotements d'une couleur, avant de passer à la suivante, dans l'ordre rouge, puis bleu, puis vert, puis retour à rouge.

Ce qu'il se déclenche le changement de couleur est visible plus précisément dans le chronogramme détaillé ci-dessous, autour de $t=20$ ms lorsque le dixième clignotement rouge se termine :



Au dernier coup d'horloge avant la fin du clignotement ($t=19\,999\,995\text{ ns}$), on observe bien l'activation du signal *end_cycle*.

Au coup d'horloge suivant ($t=20\,000\,005\text{ ns}$), alors que la LED s'éteint, le compteur de cycles *ctr_cycle* passe à 10 (« a » en hexadécimal), ce qui déclenche le signal *update_color*.

Au coup d'horloge suivant ($t=20\,000\,015\text{ ns}$), avec l'activation d'*update_color*, la FSM *Color* change d'état et passe de « red » à « blue », le signal *color_code* passe à '11'. En même temps, *ctr_cycle* est réinitialisé à 0, et le signal *update_color* repasse bas. Enfin, le signal *update_driver* en sortie du registre prend la valeur haute qu'avait *update_color*.

Au coup d'horloge suivant ($t=20\,000\,025\text{ ns}$), avec l'activation d'*update_driver*, les modules *LED_driver_bis* mettent à jour leur code couleur interne : *c_code* prend à son tour la valeur '11'. En même temps, *update_driver* reprend la dernière valeur d'*update_color*, et repasse bas.

On a encore une fois implémenté des tests automatiques dans notre testbench, pour vérifier que le comportement de notre module suit bien ce que nous attendons. Tous ces tests ont été vérifiés :

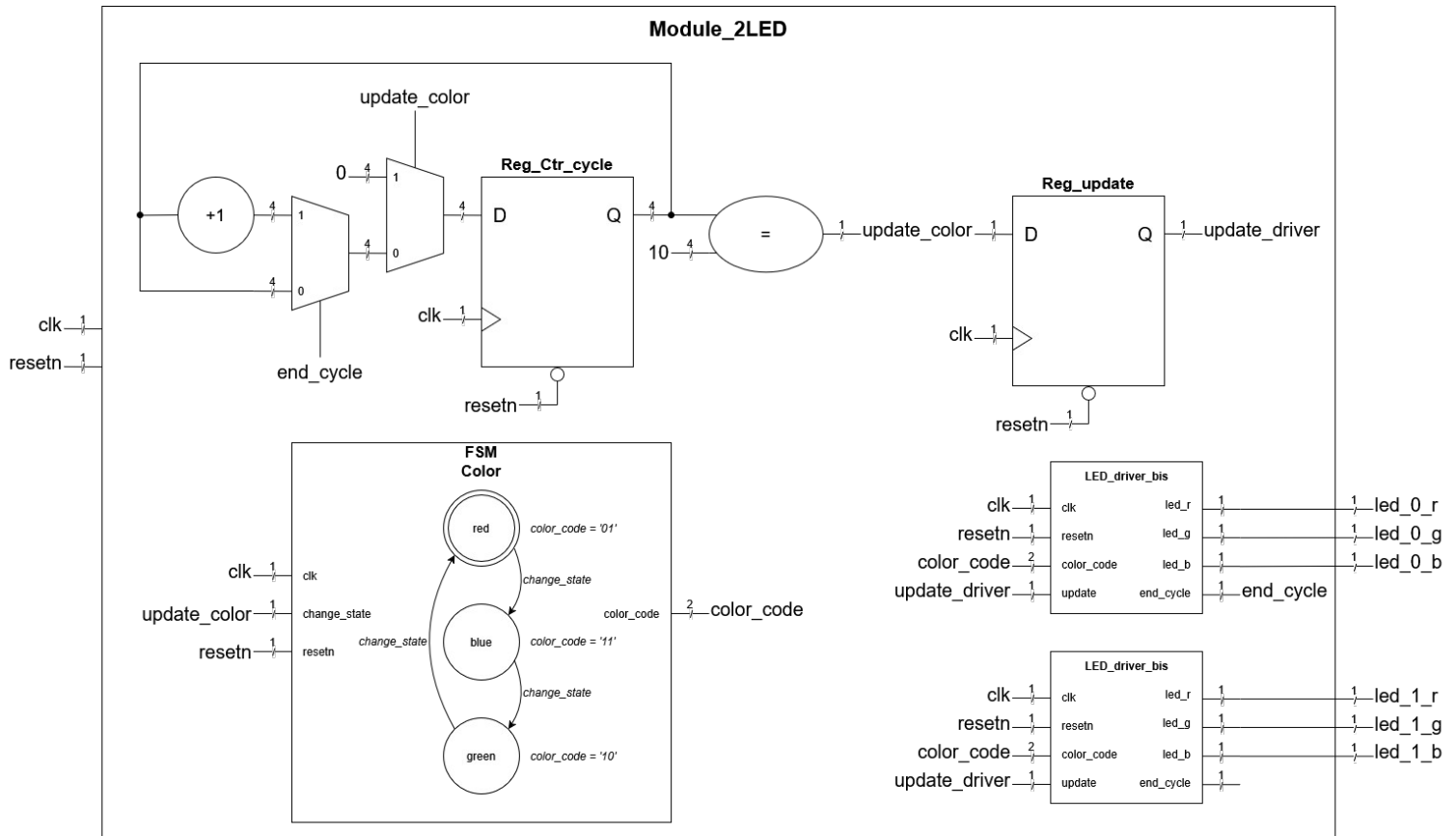
```
Note: test OK, only led_0_r = '1' at 1.000010 ms
Time: 1000010 ns Iteration: 0 Process: /tb_Module_
Note: test OK, all led = '0' at 2.000010 ms
Time: 2000010 ns Iteration: 0 Process: /tb_Module_
Note: test OK, only led_0_r = '1' at 19.000010 ms
Time: 19000010 ns Iteration: 0 Process: /tb_Module_
Note: test OK, all led = '0' at 20.000010 ms
Time: 20000010 ns Iteration: 0 Process: /tb_Module_
Note: test OK, only led_0_b = '1' at 21.000010 ms
Time: 21000010 ns Iteration: 0 Process: /tb_Module_
Note: test OK, all led = '0' at 22.000010 ms
Time: 22000010 ns Iteration: 0 Process: /tb_Module_
Note: test OK, only led_0_b = '1' at 39.000010 ms
Time: 39000010 ns Iteration: 0 Process: /tb_Module_
Note: test OK, all led = '0' at 40.000010 ms
Time: 40000010 ns Iteration: 0 Process: /tb_Module_
Note: test OK, only led_0_g = '1' at 41.000010 ms
Time: 41000010 ns Iteration: 0 Process: /tb_Module_
Note: test OK, only led_0_r = '1' at 61.000010 ms
Time: 61000010 ns Iteration: 0 Process: /tb_Module_
```

4) Nous modifions notre architecture RTL pour qu'elle utilise maintenant 2 horloges différentes :

- *clkA* qui est associée aux deux registres, à la FSM et au module *LED_driver_bis* de la LED 0 ;
- *clkB* qui est associée seulement au module *LED_driver_bis* de la LED 1.

Puisque c'est le nombre de clignotements de la LED 0 qui gère le changement de couleur, on garde le signal *end_cycle* en sortie du premier module *LED_driver_bis* pour déclencher l'incrémement du compteur de cycle.

Voici l'architecture RTL de notre nouveau module, que l'on renomme *Module_2LED_2clk* :



Nous mettons aussi à jour le code VHDL de notre module, dans le fichier *Module_2LED_2clk.vhd*.

5) En partant du testbench précédent, on en crée un nouveau, *tb_Module_2LED_2clk.vhd*.

On définit nos paramètres d'horloge :

- *clkA* a une fréquence 250 MHz, soit une période de 4 ns, avec un signal carré (demi-période 2 ns)
- *clkB* a une fréquence 50 MHz, soit une période de 20 ns, avec un signal carré (demi-période 10 ns)

On reprend le paramètre *cible* de 100000, ce qui correspond à une période de clignotement (allumé ou éteint) de 400 μ s pour *clkA* et de 2 ms pour *clkB*.

Puisque nous avons maintenant deux horloges différentes qui gèrent différentes parties de notre architecture, nous allons détecter les éventuels problèmes de « cross clock domain » en répétant 3 fois un cycle complet de fonctionnement : puisque 10 clignotements de *clkA* prennent 8 ms, on repasse en rouge au bout de 24,4 ms, et on prendra donc un cycle de 25 ms. Entre chaque cycle, on décalera nos horloge d'un coup de la plus rapide, en laissant le signal *resetn* haut.

Nous avons aussi ajusté nos tests automatiques aux timings des nouvelles horloges. Pour plus de simplicité nous avons séparés le testbench en plusieurs process :

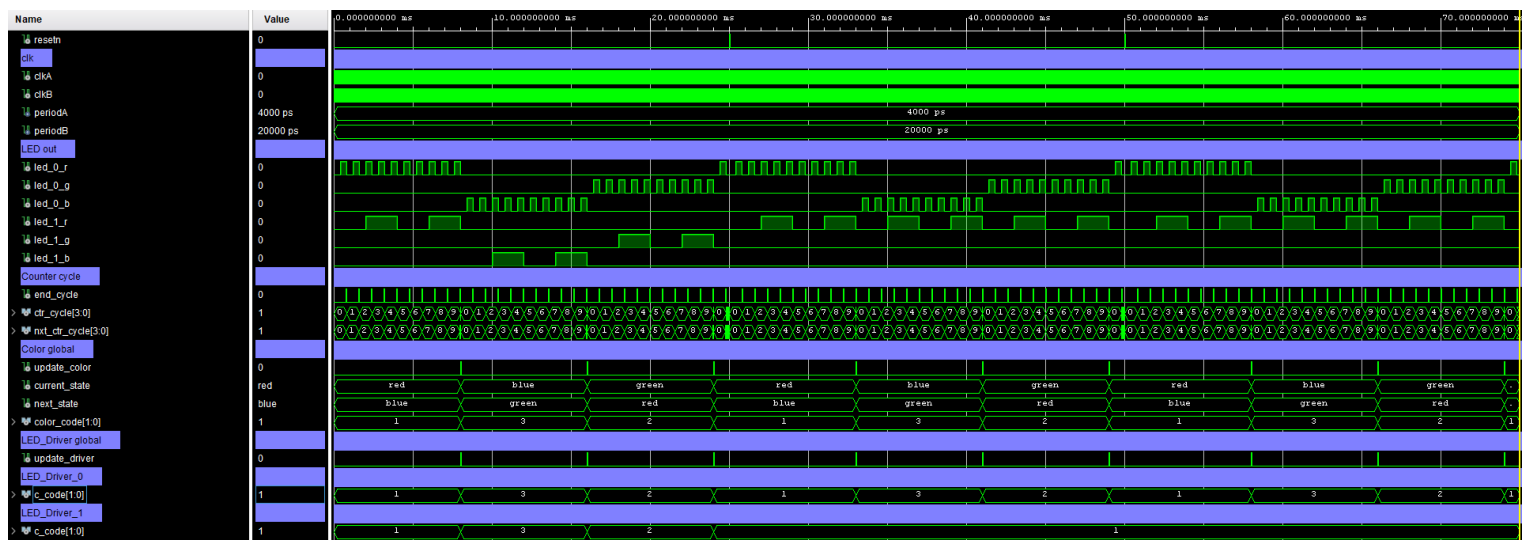
- Deux process qui génèrent chacun un des deux signaux d'horloge.
- Un process qui va actionner le signal *resetn* et créer les décalages d'horloge entre les différents cycles de la simulation.
- Un process qui effectue les tests sur les signaux de la LED 0, pilotée par l'horloge *clkA* la plus rapide. Puisque c'est la même horloge qui gère ce module *LED_driver_bis* et le reste des signaux d'entrée de ce module, on ne devrait pas avoir de problèmes de domaines d'horloge, et nos tests automatiques continuent donc à lever des « failure » en cas de manquement.
- Un process qui effectue les tests sur les signaux de la LED 1, pilotée par l'horloge *clkB* la plus lente. Puisque l'horloge qui gère ce module est différente de celle qui génère les signaux d'entrée, on peut s'attendre à des problèmes de domaines d'horloge. Pour pouvoir les observer sans que la simulation ne s'interrompe, on va relier les erreurs dans nos tests automatiques à des « warning ».

6) Pendant notre simulation, on va encore observer principalement nos six sorties qui pilotent les LEDs (*led_0_r*, *led_0_g* et *led_0_b* pour la LED 0, *led_1_r*, *led_1_g* et *led_1_b* pour la LED 1).

On va aussi observer quelques signaux internes pour vérifier le fonctionnement :

- Les signaux sur 4 bits *ctr_cycle* et *nxt_ctr_cycle* du compteur de cycles.
- Le signal *end_cycle*, en sortie du modules *LED_driver_bis_0*, qui déclenche l'incréméntation du compteur de cycle.
- Le signal *update_color* qui déclenche le changement d'état de la FSM *Color*
- Les signaux *current_state* et *next_state* qui contiennent les états (actuels et futurs) de la FSM *Color*.
- Le signal *color_code*, en sortie de la FSM *Color* et en entrée des modules *LED_driver_bis*.
- Le signal *update_driver*, qui doit lancer l'*update* des modules *LED_driver_bis*.
- Les signaux *c_code* interne aux modules *LED_driver_bis*, qui montrent si le signal *update* du module a bien été pris en compte.

On lance notre nouvelle simulation. Voici le chronogramme obtenu pour nos 3 cycles de simulation de 25 ms chacun, soit 75 ms de simulation au total :

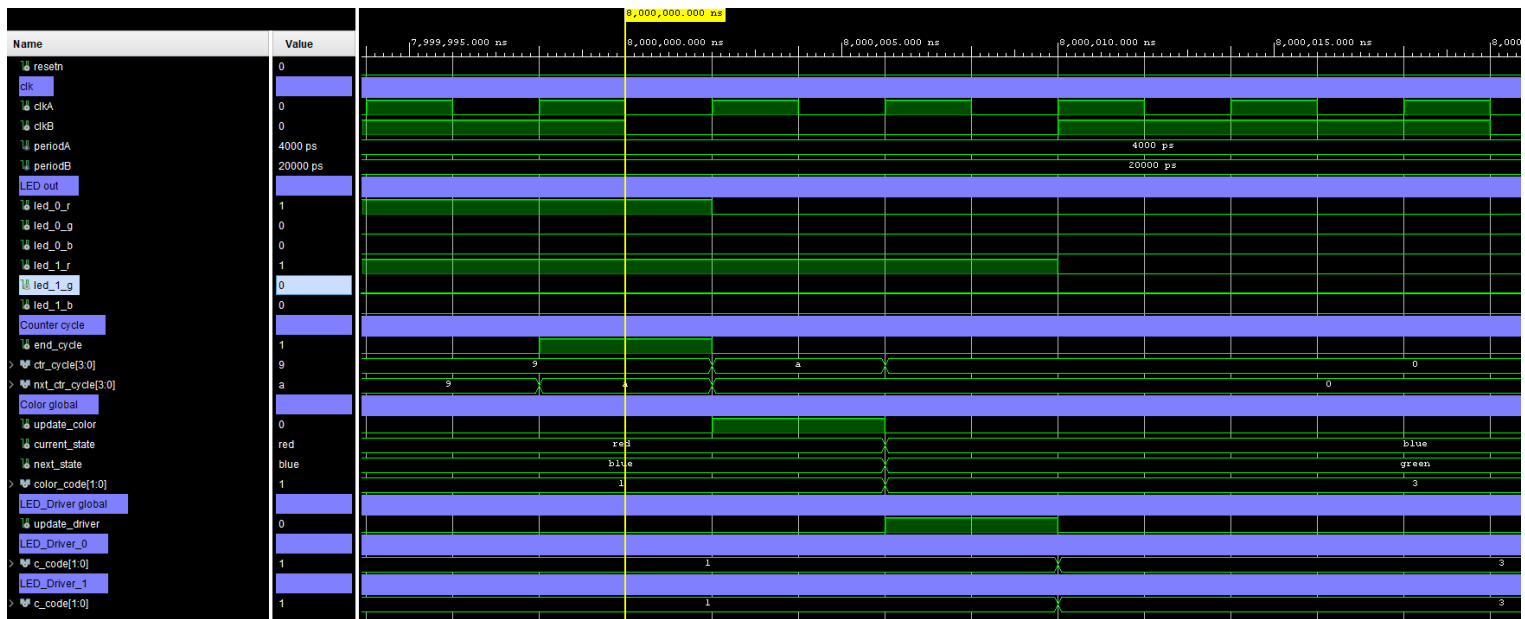


On observe que sur le premier cycle de simulation (jusqu'à $t=25$ ms), notre module à deux horloges semble bien fonctionner : les changements de couleur sur la LED 0 (sur laquelle est calé le compteur de cycle et le signal `update_color`) se répercutent bien sur la LED 1.

Cependant, sur les deux cycles suivants, le module n'a plus le fonctionnement attendu : la LED 1 clignote uniquement en rouge, les changements de couleur de la LED 0 n'ont plus de conséquence sur l'autre LED.

Pour comprendre pourquoi le premier cycle de simulation se passe correctement, mais pas les suivants, on va regarder plus en détail notre chronogramme au moment de la fin du dixième clignotement rouge de la LED 0, lorsque la couleur doit passer au bleu pour nos deux modules *LED_driver_bis*.

Voici le chronogramme détaillé autour de $t=8$ ms, lorsqu'on atteint la fin du dixième clignotement de la LED 0 dans le premier cycle de simulation :

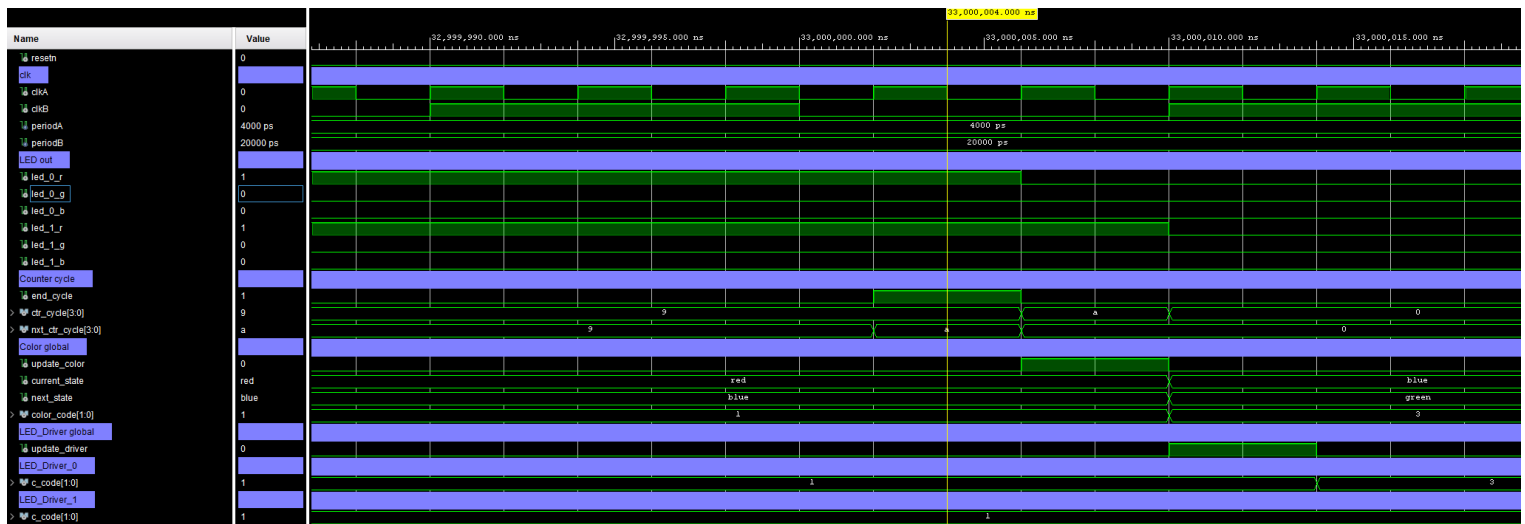


Le fonctionnement pour la LED 0 est identique à ce que l'on avait observé dans la simulation précédente (question 3), sauf pour la nouvelle fréquence d'horloge de 250 MHz au lieu de 100 MHz. Ainsi :

- à $t=7,999\,998\text{ ms}$ le signal *end_cycle* est activé ;
- au coup d'horloge suivant ($t=8,000\,002\text{ ms}$) le signal *update_color* est activé ;
- au coup d'après ($t=8,000\,006\text{ ms}$), cela lance le changement d'état de la FSM et donc la mise-à-jour de *color_code*, et parallèlement active le signal *update_driver* ;
- au coup suivant ($t=8,000\,010\text{ ms}$), l'*update* de *LED_driver_bis_0* est lancée, et le signal interne *c_code* passe au « bleu ».

Pour ce qui concerne la LED 1, l'instant $t=8,000\,010\text{ ms}$ correspond à un front montant de notre horloge *clkB*. À ce moment, le module *LED_driver_bis_1* va pouvoir prendre en compte le signal haut de son entrée *update_driver*, et lancer l'*update* du module, ce qui passe son propre signal interne *c_code* à la valeur « bleu ». La LED 1 aura bien changé de couleur à son prochain allumage.

Voici le chronogramme détaillé autour de $t=33,000\,004\text{ ms}$, lorsqu'on atteint la fin du dixième clignotement de la LED 0 dans le second cycle de simulation :



Voici le nouveau timing pour la LED 0 :

- à $t=33,000\ 002\ \text{ms}$ le signal *end_cycle* est activé ;
- au coup d'horloge suivant ($t=33,000\ 006\ \text{ms}$) le signal *update_color* est activé ;
- au coup d'après ($t=33,000\ 010\ \text{ms}$), cela lance le changement d'état de la FSM et donc la mise-à-jour de *color_code*, et parallèlement active le signal *update_driver* ;
- au coup suivant ($t=33,000\ 014\ \text{ms}$), l'*update* de *LED_driver_bis_0* est lancée, et le signal interne *c_code* passe au « bleu ».

Cette fois cependant, l'instant $t=33,000\ 014\ \text{ms}$ ne coïncide pas avec un front montant de l'horloge *clkB*. Le front montant le plus proche a lieu à $t=33,000\ 010\ \text{ms}$, avant que *update_driver* ne soit activé, et le suivant à $t=33,000\ 030\ \text{ms}$, alors que le signal *update_driver* est redevenu bas.

Sur ce cycle de simulation, il n'y a jamais de signal *update* haut en entrée de *LED_driver_bis_1* au moment d'un front montant de son horloge *clkB*. La valeur interne du module *c_code* ne sera donc jamais mise à jour et restera « rouge » pendant tout le cycle de simulation : la LED 1 ne va clignoter qu'en rouge au lieu de changer de couleur avec le LED 0.

On observe le même phénomène dans le troisième cycle de simulation : le décalage du fonctionnement sur l'horloge *clkA* fait qu'à nouveau le signal *update_driver* ne coïncidera jamais avec un front montant de *clkB*, et que la LED 1 ne changera pas de couleur.

Voici les résultats de nos tests automatiques, avec les erreurs (sous formes de warning) sur la LED 1 lors des deuxième et troisième cycles de simulation :

```

Note: test OK, only led_0_r = '1' at 0.400004 ms
Time: 400004 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 0.800004 ms
Time: 800004 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_1_r = '1' at 2.000020 ms
Time: 2000020 ns Iteration: 0 Process: /tb_Modul
Note: test OK, all led_1 = '0' at 4.000020 ms
Time: 4000020 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_1_r = '1' at 6.000020 ms
Time: 6000020 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_0_r = '1' at 7.600004 ms
Time: 7600004 ns Iteration: 0 Process: /tb_Modul
Note: test OK, all led_0 = '0' at 8.000004 ms
Time: 8000004 ns Iteration: 0 Process: /tb_Modul
Note: test OK, all led_1 = '0' at 8.000020 ms
Time: 8000020 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_0_b = '1' at 8.400004 ms
Time: 8400004 ns Iteration: 0 Process: /tb_Modul
Note: test OK, all led_0 = '0' at 8.800004 ms
Time: 8800004 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_1_b = '1' at 10.000020 ms
Time: 10000020 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_0_b = '1' at 15.600004 ms
Time: 15600004 ns Iteration: 0 Process: /tb_Modu
Note: test OK, all led_0 = '0' at 16.000004 ms
Time: 16000004 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_0_g = '1' at 16.400004 ms
Time: 16400004 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_1_g = '1' at 18.000020 ms
Time: 18000020 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_0_r = '1' at 24.400004 ms
Time: 24400004 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_0_r = '1' at 25.400008 ms
Time: 25400008 ns Iteration: 0 Process: /tb_Modu
Note: test OK, all led_0 = '0' at 25.800008 ms
Time: 25800008 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_1_r = '1' at 27.000020 ms
Time: 27000020 ns Iteration: 0 Process: /tb_Modu
Note: test OK, all led_1 = '0' at 29.000020 ms
Time: 29000020 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_1_r = '1' at 31.000020 ms
Time: 31000020 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_0_r = '1' at 32.600008 ms
Time: 32600008 ns Iteration: 0 Process: /tb_Modu
Note: test OK, all led_0 = '0' at 33.000008 ms
Time: 33000008 ns Iteration: 0 Process: /tb_Modu
Note: test OK, all led_1 = '0' at 33.000020 ms
Time: 33000020 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_0_b = '1' at 33.400008 ms
Time: 33400008 ns Iteration: 0 Process: /tb_Modu
Note: test OK, all led_0 = '0' at 33.800008 ms
Time: 33800008 ns Iteration: 0 Process: /tb_Modu

```

```

Warning: test failed - led_1_r != '0' at 35.000020 ms
Time: 35000020 ns Iteration: 0 Process: /tb_Module_2
Warning: test failed - led_1_b != '1' at 35.000020 ms
Time: 35000020 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, only led_0_b = '1' at 40.600008 ms
Time: 40600008 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, all led_0 = '0' at 41.000008 ms
Time: 41000008 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, only led_0_g = '1' at 41.400008 ms
Time: 41400008 ns Iteration: 0 Process: /tb_Module_2
Warning: test failed - led_1_r != '0' at 43.000020 ms
Time: 43000020 ns Iteration: 0 Process: /tb_Module_2
Warning: test failed - led_1_g != '1' at 43.000020 ms
Time: 43000020 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, only led_0_r = '1' at 49.400008 ms
Time: 49400008 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, only led_0_r = '1' at 50.400012 ms
Time: 50400012 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, all led_0 = '0' at 50.800012 ms
Time: 50800012 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, only led_1_r = '1' at 52.000020 ms
Time: 52000020 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, all led_1 = '0' at 54.000020 ms
Time: 54000020 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, only led_1_r = '1' at 56.000020 ms
Time: 56000020 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, only led_0_r = '1' at 57.600012 ms
Time: 57600012 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, all led_0 = '0' at 58.000012 ms
Time: 58000012 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, all led_1 = '0' at 58.000020 ms
Time: 58000020 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, only led_0_b = '1' at 58.400012 ms
Time: 58400012 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, all led_0 = '0' at 58.800012 ms
Time: 58800012 ns Iteration: 0 Process: /tb_Module_2
Warning: test failed - led_1_r != '0' at 60.000020 ms
Time: 60000040 ns Iteration: 0 Process: /tb_Module_2
Warning: test failed - led_1_b != '1' at 60.000020 ms
Time: 60000040 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, only led_0_b = '1' at 65.600012 ms
Time: 65600012 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, all led_0 = '0' at 66.000012 ms
Time: 66000012 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, only led_0_g = '1' at 66.400012 ms
Time: 66400012 ns Iteration: 0 Process: /tb_Module_2
Warning: test failed - led_1_r != '0' at 68.000020 ms
Time: 68000040 ns Iteration: 0 Process: /tb_Module_2
Warning: test failed - led_1_g != '1' at 68.000020 ms
Time: 68000040 ns Iteration: 0 Process: /tb_Module_2
Note: test OK, only led_0_r = '1' at 74.400012 ms

```

7) Comme on l'a vu dans la simulation précédente, notre problème de changement de domaine d'horloge provient de l'utilisation par une horloge lente d'un signal déclencheur produit par une horloge plus rapide.

En reprenant les suggestions présentées sur le lien <https://nandland.com/lesson-14-crossing-clock-domains/>, nous pouvons proposer deux solutions à notre problème :

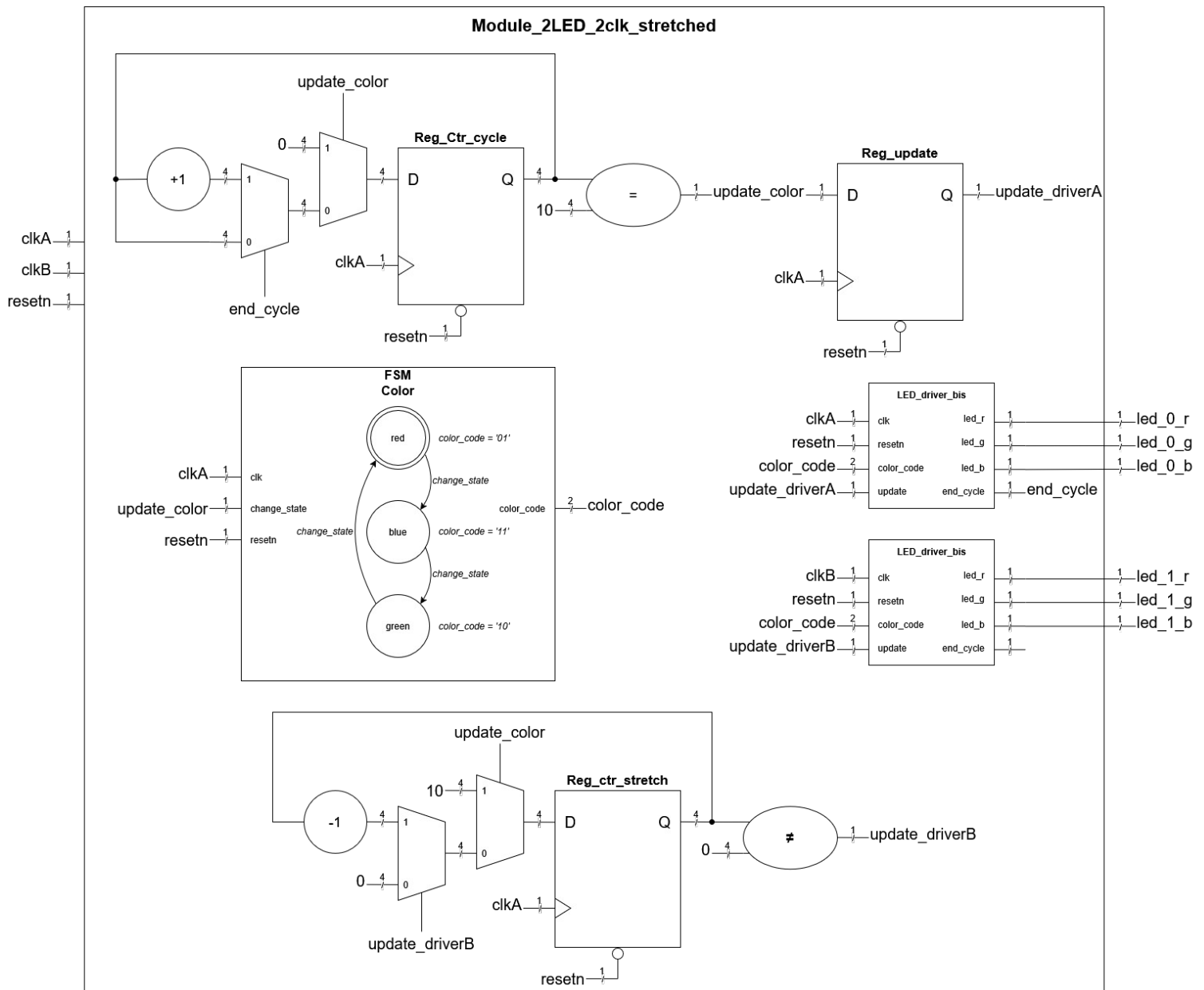
- Élargir notre signal *update_driver* pour être sûr qu'il est détecté par un front montant de l'horloge *clkB*.
- Utiliser une FIFO avec deux horloges différentes pour l'écriture et à la lecture.

Dans tous les cas, on va renommer le signal *update_driver* en *update_driverA*, et utiliser ce signal uniquement en entrée *update* du premier module *LED_driver_bis*. Le signal en entrée *update* du second module *LED_driver_bis* sera nommé *update_driverB*.

Pour la première solution, nous voulons élargir le signal à l'entrée *update* du module *LED_Driver_bis* de la LED 1, pour qu'il soit forcément détecté par un front montant de l'horloge *clkB*. Pour cela, on choisit de le faire durer deux cycles de l'horloge *clkB*, soit 10 cycles de l'horloge *clkA*.

On ajoute un compteur « stretch », stocké sur 4 bits de registres synchronisés sur l'horloge *clkA*. Le compteur est initialisé à 10 lorsqu'on reçoit le signal *update_color*, décrémente de 1 à chaque coup d'horloge et se maintient à 0 lorsque cette valeur est atteinte. Le signal *update_driverB* est constitué par logique conditionnel, il est haut quand la valeur du compteur est différent de 0. C'est ce signal qui est utilisé comme entrée *update* pour le deuxième module *LED_driver_bis*.

Voici l'architecture RTL du module modifié pour mettre en place cette première solution :



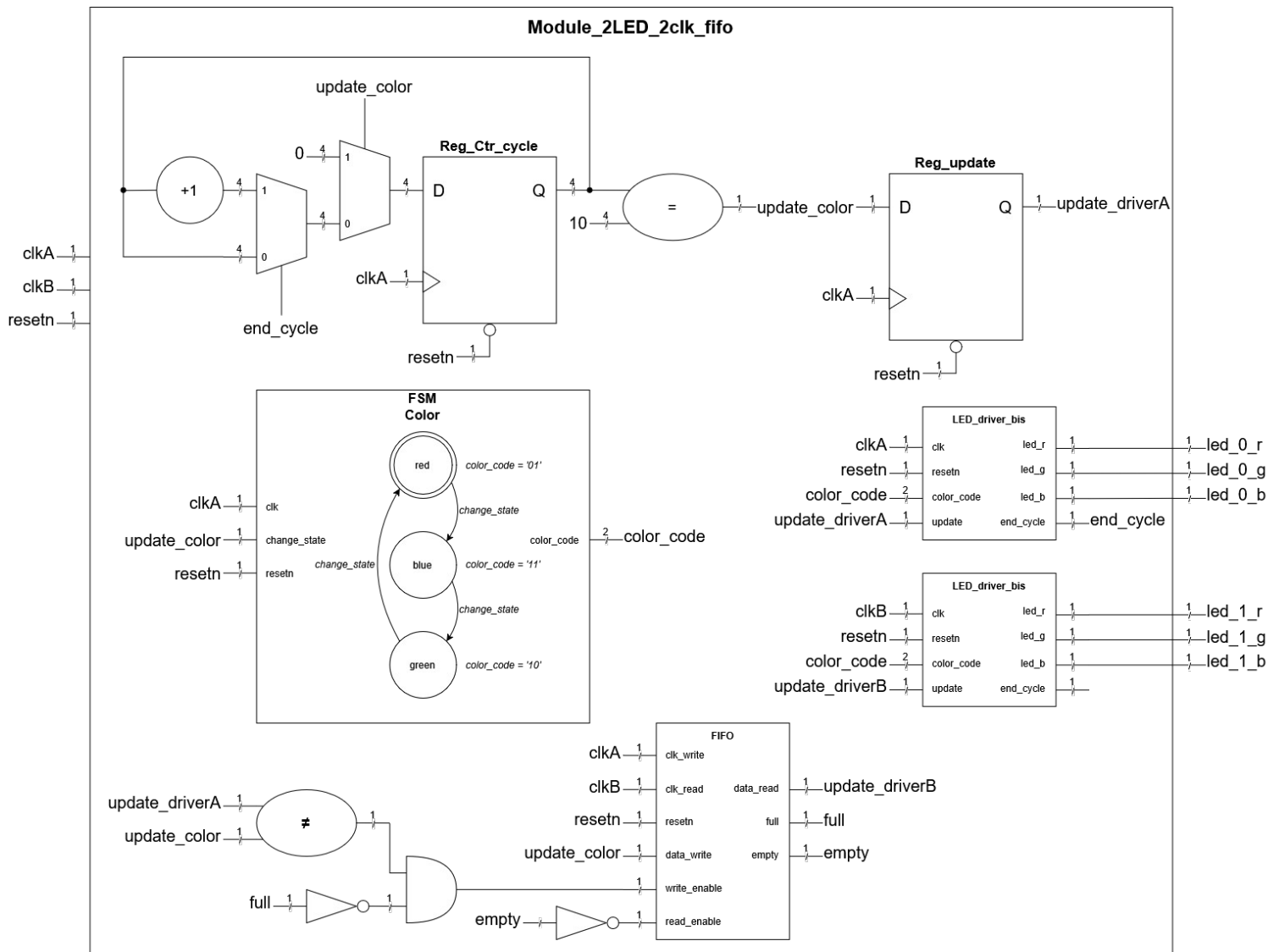
On pourra aussi remarquer que le signal *color_code* est un signal dont la valeur, produite par une FSM, est donnée en continu avec des moments de mise-à-jour compatible avec le fonctionnement voulu de notre architecture. Si la valeur d'*update* des deux modules *LED_driver_bis* étaient toujours haute, le fonctionnement resterait inchangé, et nous éviterions les problèmes de domaine d'horloge, tout en évitant d'avoir un registre devant gérer le décalage entre le signal *update_color* et *update_driverA*. Cependant, le but du TP étant justement de gérer les domaines d'horloges, nous ne retiendrons pas cette solution.

Pour la deuxième solution, nous allons utiliser une FIFO avec deux horloges différentes en écriture et en lecture, avec 1 bit de donnée. Nous allons faire en sorte que la FIFO, au moment où on veut déclencher une *update* du deuxième module *LED_driver_bis*, envoie un signal '1' pendant un coup d'horloge *clkB*, puis renvoie à nouveau '0' jusqu'à la prochaine *update*.

Côté écriture, l'horloge d'entrée est *clkA*. Aux moment de la mise à jour du code couleur, nous voulons donner à la FIFO deux signaux successifs, d'abord '1' puis '0'. Pour cela, on va connecter comme donnée d'entrée le signal *update_color*, en autorisant l'écriture dans la FIFO lorsque la FIFO n'est pas pleine et que *update_color* est différent de *update_driverA* : puisque *update_driverA* est un décalage d'un coup d'horloge de *update_color*, et que ce dernier est une impulsion, l'entrée en écriture de la FIFO vaudra bien '1' puis '0'.

Côté lecture, l'horloge d'entrée est *clkB*, la donnée de sortie est *update_driverB*, et la condition de lecture est que la FIFO ne soit pas vide.

Voici l'architecture RTL du module modifié pour mettre en place cette deuxième solution :



8) Nous implémentons en VHDL les deux solutions proposées :

- La solution d'élargissement du signal *update_driver* est implémentée dans le fichier *Module_2LED_2clk_stretched.vhd*. La mise-à-jour et le reset du nouveau compteur sont gérés dans un nouveau process de la partie séquentielle, la décrémentation et la réinitialisation du compteur est elle ajoutée à la partie combinatoire.
- La solution d'ajout d'une FIFO est implémentée dans le fichier *Module_2LED_2clk_fifo.vhd*. La FIFO est générée à nouveau avec l'IP *fifo_generator* de Xilinx, avec les paramètres résumés dans le tableau ci-dessous. Les conditions d'écriture et de lecture sont ajoutées à la partie combinatoire du code.

Estimated FIFO Usage : 1 of type: 512x36

Number of FIFO chained in depth : 1

Clocking Scheme	Independent Clocks
Memory Type	Built-in FIFO
Model Generated	Behavioral Model
Write Width	1
Write Depth	512
Read Width	1
Read Depth	512
Almost Full/Empty Flags	Not Selected/Not Selected
Programmable Full/Empty Flags	Not Selected/Not Selected
Data Count Outputs	Not Selected
Handshaking	Not Selected
Read Mode / Reset	Standard FIFO / Asynchronous
Read Latency (From Rising Edge of Read Clock)	1

Ces deux solutions vont être testées dans deux testbench, qui reprennent les tests de la question 6. Ces testbenches sont dans les fichiers *tb_Module_2LED_2clk_stretched.vhd* et *tb_Module_2LED_2clk_fifo.vhd*.

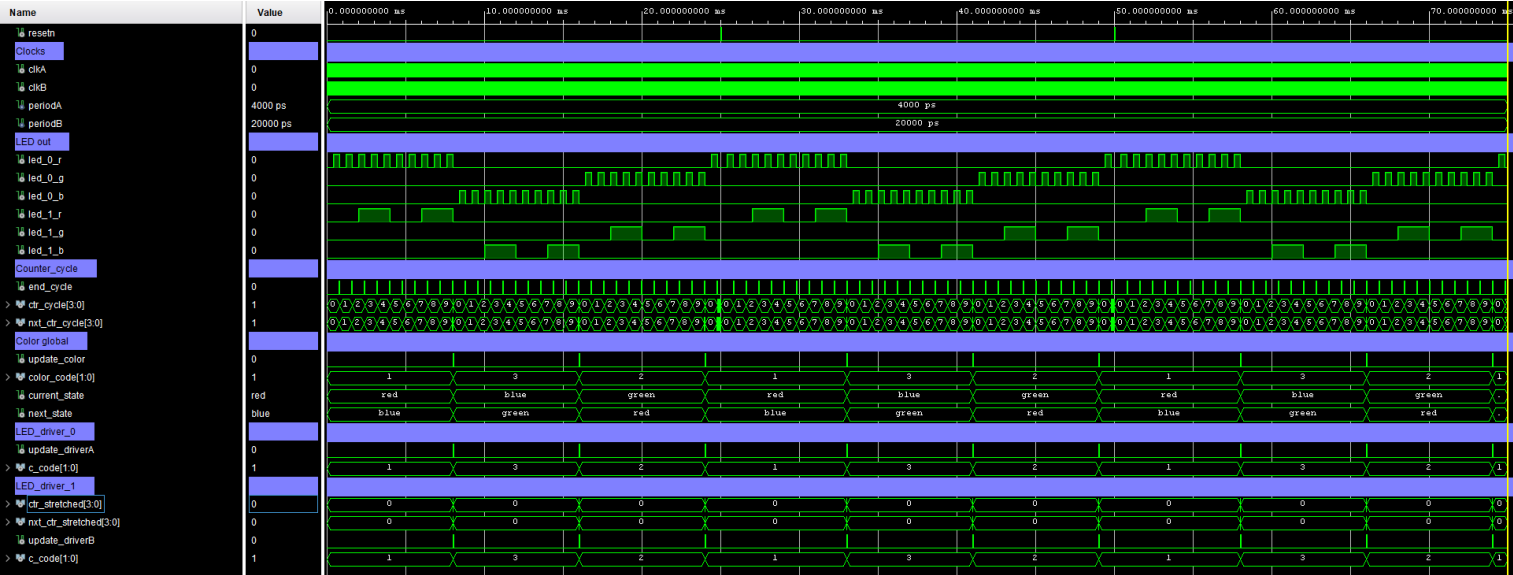
On garde les mêmes paramètres de simulation : deux horloges, *clkA* a une fréquence 250 MHz et *clkB* a une fréquence 50 MHz, soit une période de 20 ns, avec un signal carré (demi-période 10 ns), et un paramètre *cible* de 100000, ce qui correspond à une période de clignotement (allumé ou éteint) de 400 µs pour *clkA* et de 2 ms pour *clkB*.

Pendant nos simulations, on va encore observer principalement nos six sorties qui pilotent les LEDs (*led_0_r*, *led_0_g* et *led_0_b* pour la LED 0, *led_1_r*, *led_1_g* et *led_1_b* pour la LED 1). On va aussi observer quelques signaux internes pour vérifier le fonctionnement :

- Les signaux sur 4 bits *ctr_cycle* et *nxt_ctr_cycle* du compteur de cycles.
- Le signal *end_cycle*, en sortie du modules *LED_driver_bis_0*, qui déclenche l'incréméntation du compteur de cycle.
- Le signal *update_color* qui déclenche le changement d'état de la FSM *Color*
- Les signaux *current_state* et *next_state* qui contiennent les états (actuels et futurs) de la FSM *Color*.
- Le signal *color_code*, en sortie de la FSM *Color* et en entrée des modules *LED_driver_bis*.
- Les signaux *update_driverA* et *update_driverB*, qui doivent lancer l'*update* des modules *LED_driver_bis*.
- Les signaux *c_code* internes aux modules *LED_driver_bis*, qui montrent si le signal *update* du module a bien été pris en compte.

Pour la simulation du testbench *tb_Module_2LED_2clk_stretched.vhd*, on va aussi observer les signaux qui réalisent l'étirement du signal *update_driverB*, c'est à dire *ctr_stretched* et *nxt_ctr_stretched*.

Voici le chronogramme obtenu sur ce premier testbench, pour nos 3 cycles de simulation de 25 ms chacun, soit 75 ms de simulation au total :



De manière générale, on peut observer que nos deux LEDs suivent bien le motif de couleurs attendu sur les 3 cycles de simulation. Ce premier correctif semble fonctionner.

Pour comprendre en détail l’effet de l’allongement du signal *update_driverB*, on peut observer les chronogrammes détaillés autour de t=8 ms et t=33,000 004 ms, lorsqu’on atteint la fin du dixième clignotement de la LED 0 dans le premier et le second cycle de simulation :





On remarque que le compteur « stretch » que nous avons implémenté rempli bien son rôle : le signal *update_driverB* n'est plus une impulsion d'un seul coup d'horloge *clkA*, mais de 10 coups d'horloge *clkA* (ou 2 coups de *clkB*). Ainsi, le module *LED_driver_bis_1* peut détecter le signal haut de *update_driverB* lors d'au moins un front haut de *clkB*, et peut mettre à jour son code couleur un peu après *LED_driver_bis_0*. Cela continue à marcher même quand on décale le fonctionnement sur l'horloge *clkA* par rapport à *clkB*.

Voici les résultats de nos tests automatiques pour cette première simulation, où aucune erreur ou warning n'ont été soulevé :


```

Note: test OK, only led_0_r = '1' at 0.400004 ms
Time: 400004 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 0.800004 ms
Time: 800004 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_r = '1' at 2.000020 ms
Time: 2000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_l = '0' at 4.000020 ms
Time: 4000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_r = '1' at 6.000020 ms
Time: 6000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_r = '1' at 7.600004 ms
Time: 7600004 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 8.000004 ms
Time: 8000004 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_l = '0' at 8.000020 ms
Time: 8000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_b = '1' at 8.400004 ms
Time: 8400004 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 8.800004 ms
Time: 8800004 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_b = '1' at 10.000020 ms
Time: 10000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_b = '1' at 15.600004 ms
Time: 15600004 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 16.000004 ms
Time: 16000004 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_g = '1' at 16.400004 ms
Time: 16400004 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_g = '1' at 18.000020 ms
Time: 18000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_r = '1' at 24.400004 ms
Time: 24400004 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_r = '1' at 25.400008 ms
Time: 25400008 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 25.800008 ms
Time: 25800008 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_r = '1' at 27.000020 ms
Time: 27000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_l = '0' at 29.000020 ms
Time: 29000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_r = '1' at 31.000020 ms
Time: 31000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_r = '1' at 32.600008 ms
Time: 32600008 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 33.000008 ms
Time: 33000008 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_l = '0' at 33.000020 ms
Time: 33000020 ns Iteration: 0 Process: /tb_Module

```

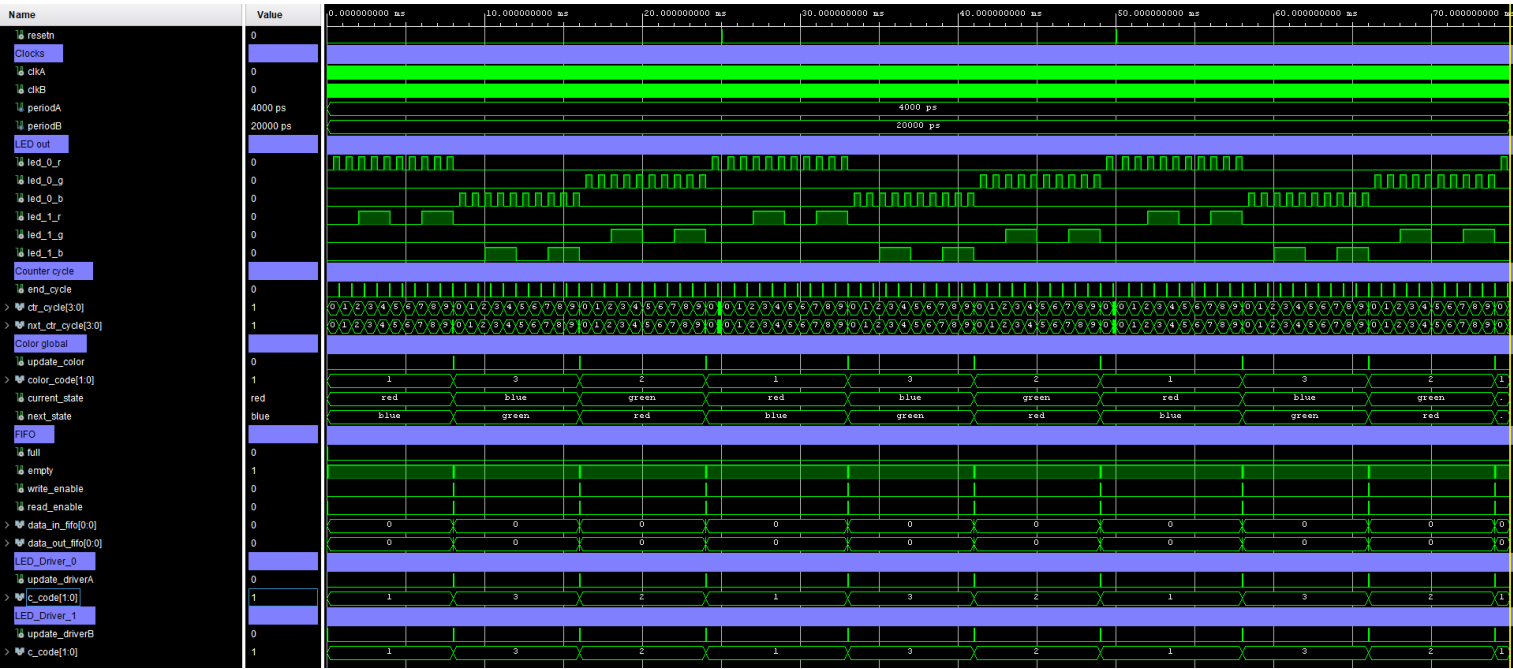
```

Note: test OK, only led_0_b = '1' at 33.400008 ms
Time: 33400008 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 33.800008 ms
Time: 33800008 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_b = '1' at 35.000020 ms
Time: 35000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_b = '1' at 40.600008 ms
Time: 40600008 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 41.000008 ms
Time: 41000008 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_g = '1' at 41.400008 ms
Time: 41400008 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_g = '1' at 43.000020 ms
Time: 43000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_r = '1' at 49.400008 ms
Time: 49400008 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_r = '1' at 50.400012 ms
Time: 50400012 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 50.800012 ms
Time: 50800012 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_r = '1' at 52.000020 ms
Time: 52000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_l = '0' at 54.000020 ms
Time: 54000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_r = '1' at 56.000020 ms
Time: 56000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_r = '1' at 57.600012 ms
Time: 57600012 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 58.000012 ms
Time: 58000012 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_l = '0' at 58.000020 ms
Time: 58000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_b = '1' at 58.400012 ms
Time: 58400012 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 58.800012 ms
Time: 58800012 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_b = '1' at 60.000020 ms
Time: 60000040 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_b = '1' at 65.600012 ms
Time: 65600012 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 66.000012 ms
Time: 66000012 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_g = '1' at 66.400012 ms
Time: 66400012 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_g = '1' at 68.000020 ms
Time: 68000040 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_r = '1' at 74.400012 ms
Time: 74400012 ns Iteration: 0 Process: /tb_Module

```

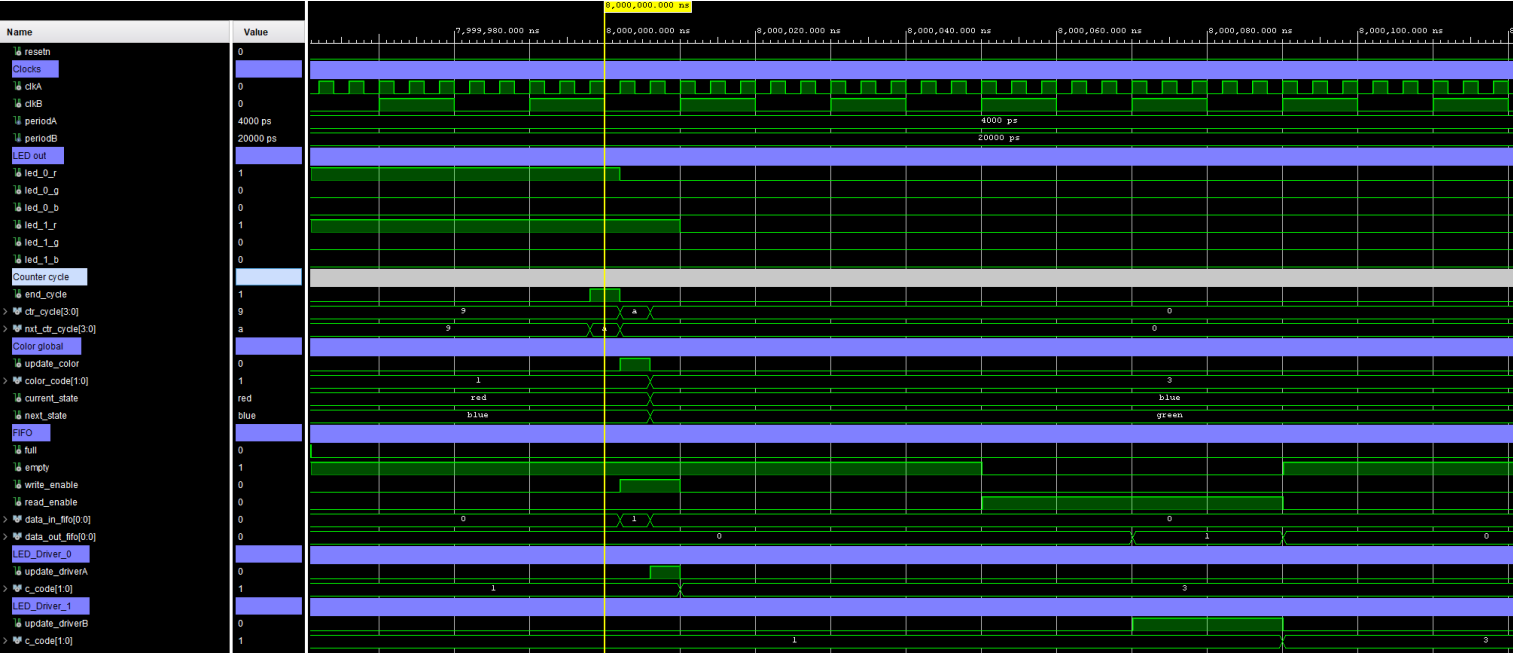
Pour la seconde simulation, sur le testbench *tb_Module_2LED_2clk_fifo.vhd*, on va également observer les signaux de la FIFO : les données en entrée/sortie *data_in_fifo* et *data_out_fifo*, les autorisations d'écriture/lecture *write_enable* et *read_enable*, et les indications de remplissage *full* et *empty*.

Voici le chronogramme obtenu sur ce second testbench, pour nos 3 cycles de simulation de 25 ms chacun, soit 75 ms de simulation au total :



De manière générale, on peut observer à nouveau que nos deux LEDs suivent bien le motif de couleurs attendu sur les 3 cycles de simulation.

Pour comprendre en détail l’effet de notre FIFO pour créer notre signal *update_driverB*, on peut observer le chronogramme détaillé autour de t=8 ms, lorsqu’on atteint la fin du dixième clignotement de la LED 0 dans le premier cycle de simulation :



Pendant le coup d’horloge où *update_color* est haut, ainsi que le suivant, le signal *write_enable* est activé. Le signal de données à l’entrée de la FIFO vaut pendant ces deux coups d’horloge ‘1’ puis ‘0’. Ces deux signaux sont enregistrés dans la FIFO, le processus d’enregistrement durant jusqu’à t=8,000 050 ms. À ce moment, le signal *empty* passe à ‘0’, signalant que la FIFO n’est plus vide, ce qui active l’autorisation de lecture *read_enable*.

Au coup d'horloge suivant de *clkB* (t=8,000 070 ms), la sortie de la FIFO a été mise à jour avec la valeur '1', ce qui active le signal *update_driverB*.

Cela permet au coup d'horloge suivant de *clkB* (t=8,000 090 ms) au module *LED_Driver_bis_1* de se mettre-à-jour, le signal interne *c_code* prenant la valeur « bleu ». En parallèle, la FIFO lit le signal suivant dans la file, sa sortie et le signal *update_driverB* passe à '0', évitant de renouveler inutilement la mise-à-jour du module *LED_driver_bis_1*. Le signal *empty* de la FIFO repasse à '1' : il n'y a plus de nouvelle lecture dans la FIFO jusqu'à ce qu'on écrive à nouveau dedans.

Voici les résultats de nos tests automatiques pour cette seconde simulation, où à nouveau aucune erreur ou warning n'ont été soulevé :

```
Note: test OK, only led_0_r = '1' at 0.400004 ms
Time: 400004 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 0.800004 ms
Time: 800004 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_r = '1' at 2.000020 ms
Time: 2000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_l = '0' at 4.000020 ms
Time: 4000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_r = '1' at 6.000020 ms
Time: 6000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_r = '1' at 7.600004 ms
Time: 7600004 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 8.000004 ms
Time: 8000004 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_l = '0' at 8.000020 ms
Time: 8000020 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_0_b = '1' at 8.400004 ms
Time: 8400004 ns Iteration: 0 Process: /tb_Module
Note: test OK, all led_0 = '0' at 8.800004 ms
Time: 8800004 ns Iteration: 0 Process: /tb_Module
Note: test OK, only led_l_b = '1' at 10.000020 ms
Time: 10000020 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_0_b = '1' at 15.600004 ms
Time: 15600004 ns Iteration: 0 Process: /tb_Modu
Note: test OK, all led_0 = '0' at 16.000004 ms
Time: 16000004 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_0_g = '1' at 16.400004 ms
Time: 16400004 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_l_g = '1' at 18.000020 ms
Time: 18000020 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_0_r = '1' at 24.400004 ms
Time: 24400004 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_0_r = '1' at 25.400008 ms
Time: 25400008 ns Iteration: 0 Process: /tb_Modu
Note: test OK, all led_0 = '0' at 25.800008 ms
Time: 25800008 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_l_r = '1' at 27.000020 ms
Time: 27000020 ns Iteration: 0 Process: /tb_Modu
Note: test OK, all led_l = '0' at 29.000020 ms
Time: 29000020 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_l_r = '1' at 31.000020 ms
Time: 31000020 ns Iteration: 0 Process: /tb_Modu
Note: test OK, only led_0_r = '1' at 32.600008 ms
Time: 32600008 ns Iteration: 0 Process: /tb_Modu
Note: test OK, all led_0 = '0' at 33.000008 ms
Time: 33000008 ns Iteration: 0 Process: /tb_Modu
Note: test OK, all led_l = '0' at 33.000020 ms
Time: 33000020 ns Iteration: 0 Process: /tb_Modu
```

```
Note: test OK, only led_0_b = '1' at 33.400008 ms
Time: 33400008 ns Iteration: 0 Process: /tb_Modul
Note: test OK, all led_0 = '0' at 33.800008 ms
Time: 33800008 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_l_b = '1' at 35.000020 ms
Time: 35000020 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_0_b = '1' at 40.600008 ms
Time: 40600008 ns Iteration: 0 Process: /tb_Modul
Note: test OK, all led_0 = '0' at 41.000008 ms
Time: 41000008 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_0_g = '1' at 41.400008 ms
Time: 41400008 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_l_g = '1' at 43.000020 ms
Time: 43000020 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_0_r = '1' at 49.400008 ms
Time: 49400008 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_0_r = '1' at 50.400012 ms
Time: 50400012 ns Iteration: 0 Process: /tb_Modul
Note: test OK, all led_0 = '0' at 50.800012 ms
Time: 50800012 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_l_r = '1' at 52.000020 ms
Time: 52000020 ns Iteration: 0 Process: /tb_Modul
Note: test OK, all led_l = '0' at 54.000020 ms
Time: 54000020 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_l_r = '1' at 56.000020 ms
Time: 56000020 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_0_r = '1' at 57.600012 ms
Time: 57600012 ns Iteration: 0 Process: /tb_Modul
Note: test OK, all led_0 = '0' at 58.000012 ms
Time: 58000012 ns Iteration: 0 Process: /tb_Modul
Note: test OK, all led_l = '0' at 58.000020 ms
Time: 58000020 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_0_b = '1' at 58.400012 ms
Time: 58400012 ns Iteration: 0 Process: /tb_Modul
Note: test OK, all led_0 = '0' at 58.800012 ms
Time: 58800012 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_l_b = '1' at 60.000020 ms
Time: 60000040 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_0_b = '1' at 65.600012 ms
Time: 65600012 ns Iteration: 0 Process: /tb_Modul
Note: test OK, all led_0 = '0' at 66.000012 ms
Time: 66000012 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_0_g = '1' at 66.400012 ms
Time: 66400012 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_l_g = '1' at 68.000020 ms
Time: 68000040 ns Iteration: 0 Process: /tb_Modul
Note: test OK, only led_0_r = '1' at 74.400012 ms
Time: 74400012 ns Iteration: 0 Process: /tb_Modul
```

9) L'IP Vivado pour appeler une PLL est « Clocking Wizard ».

Voici les paramètres que j'ai utilisés pour créer les horloges pour notre architecture :

Primary Input Clock Attributes

Input Clock Frequency (MHz)	125.000
Clock Source	Single_ended_clock_capable_pin
Jitter	0.010

Clocking Primitive Attributes

Primitive Instantiated : PLL

Divide Counter : 1

Mult Counter : 8

Clock Phase Shift : None

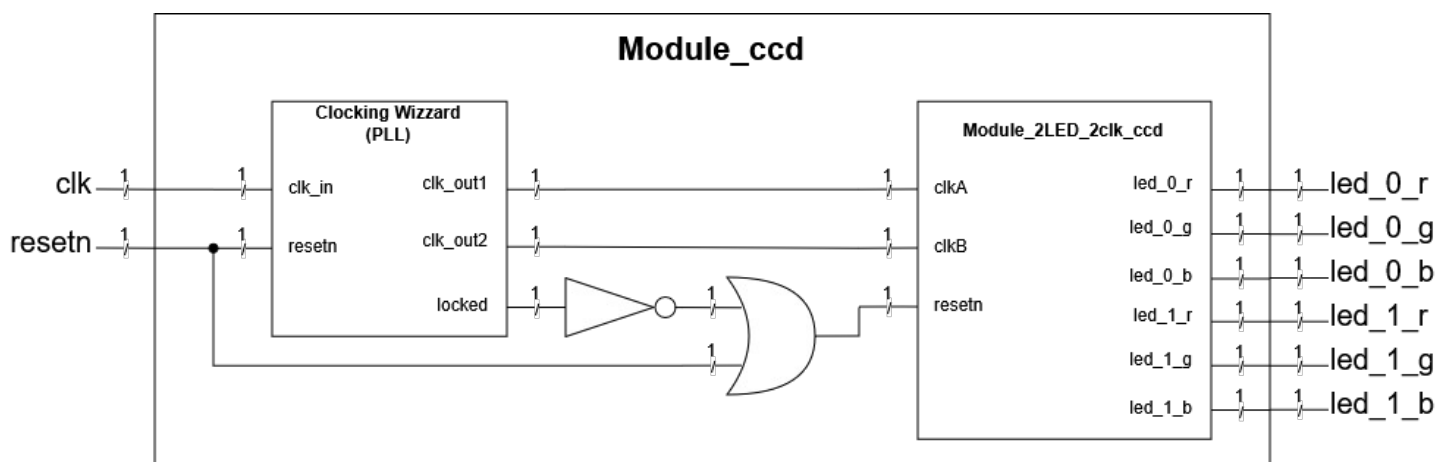
Clock Wiz O/p Pins	Source	Divider Value	Tspread (ps)	Pk-to-Pk Jitter (ps)	Phase Error (ps)
clk_out1	PLL CLKOUT0	4	OFF	104.759	96.948
clk_out2	PLL CLKOUT1	20	OFF	143.688	96.948
clk_out3	OFF	OFF	OFF	OFF	OFF
clk_out4	OFF	OFF	OFF	OFF	OFF
clk_out5	OFF	OFF	OFF	OFF	OFF
clk_out6	OFF	OFF	OFF	OFF	OFF
clk_out7	OFF	OFF	OFF	OFF	OFF

Pour notre architecture finale, nous pourrons utiliser indifféremment une des deux solutions que nous avons décrites à la question 7, c'est à dire *Module_2LED_2clk_stretched* ou *Module_2LED_2clk_fifo*. Pour simplifier leur notification sur le schéma d'architecture final, nous appellerons de manière général le module utilisé *Module_2LED_2clk_ccd*.

Pour créer notre architecture finale, nous relierons l'entrée de la PLL à l'horloge *clk*, qui sera lié dans notre fichier de contraintes à l'horloge externe de notre carte CoraZ7. Les horloges de sorties de notre PLL sont liées aux entrées d'horloge *clkA* et *clkB* de notre module *Module_2LED_2clk_ccd*.

Pour s'assurer que les horloges créées par notre PLL sont stabilisées avant d'être utilisées, nous conditionnons le fonctionnement du module *Module_2LED_2clk_ccd* en reliant son entrée *resetrn* par un OR avec le signal externe *resetrn* et la sortie *locked* de la PLL après un NOT.

Voici le schéma RTL de notre architecture finale, qu'on nommera *Module_ccd* :



10) Nous implémentons en code VHDL notre module final dans le fichier *Module_ccd.vhd*.

En plus de l'instanciation de nos sous-module (« Clocking Wizard » et *Module_2LED_2clk_ccd*), une partie combinatoire calcul le signal *resetsn_ccd* en entrée de *Module_2LED_2clk_ccd*.

Nous réaliserons une première synthèse de notre module en utilisant *Module_2LED_2clk_stretched*, puis une seconde en utilisant *Module_2LED_2clk_fifo*.

11) Analysons les rapports obtenus pour nos deux synthèses.

De manière générale, on verra dans les deux rapports :

```
INFO: [Synth 8-802] inferred FSM for state register 'current_state_reg' in module 'Module_2LED_2clk_stretched'
-----
      State |           New Encoding |           Previous Encoding
-----
      red   |           001 |           00
      blue  |           010 |           10
      green |           100 |           01
-----
INFO: [Synth 8-3354] encoded FSM with state register 'current_state_reg' using encoding 'one-hot' in module 'Module_2LED_2clk_stretched'
```

Ainsi, les états de notre FSM *Color* sont implémentés en utilisant l'encodage « one-hot » où chaque état possible est codé sur son propre bit. C'est très adapté dans notre cas où les états cycles dans un ordre précis, ce qui permet de connaître l'état suivant d'un bit uniquement en connaissant l'état actuel du bit (si l'état ne change pas) et celui du bit « précédent » (lorsque l'état change) en ignorant totalement l'état du troisième bit.

Pour la première synthèse (avec *Module_2LED_2clk_stretched*), le rapport de synthèse est dans le fichier *Module_ccd_stretched.vds*. Il nous donne les ressources utilisées suivantes :

```
Report BlackBoxes:
+-----+-----+-----+
|      |BlackBox name |Instances |
+-----+-----+-----+
|1      |clk_wiz_0     |1|
+-----+-----+-----+

Report Cell Usage:
+-----+-----+-----+
|      |Cell          |Count |
+-----+-----+-----+
|1      |clk_wiz_0_bbox |1|
|2      |CARRY4        |14|
|3      |LUT1          |2|
|4      |LUT2          |1|
|5      |LUT3          |7|
|6      |LUT4          |41|
|7      |LUT5          |13|
|8      |LUT6          |7|
|9      |FDCE          |69|
|10     |FDPE          |3|
|11     |IBUF          |1|
|12     |OBUF          |6|
+-----+-----+-----+
```

La schématique détaillée de cette synthèse est disponible dans le fichier *Schematic_TP5_stretched.pdf*.

Parmi les nouvelles fonctionnalités mises en places pendant ce TP, on pourra remarquer :

- Il faut 1 IBUF pour l'entrée *resetsn* et 6 OBUF pour les 6 canaux de sorties LED.
À noter : l'entrée *clk* étant maintenant reliée à la PLL, elle n'a plus besoin ni de IBUF, ni de BUFG.
- Le calcul du *resetsn_ccd* est effectué dans le sous-groupe *Counter_unit* de *LED_driver_bis_0*, par une LUT2 qui effectue le OR entre *resetsn* et le signal *locked* de la PPL.
- Le compteur de cycles à un fonctionnement un peu simplifié par rapport à celui qu'on a pu observer pour *Counter_unit* dans les TP précédents. Le compteur n'a que 4 bits, il n'a donc pas besoin de CARRY4, chaque bit va être stocké par un registre FDCE, une LUT calcule la prochaine valeur du bit (incrémentement/réinitialisation) en fonction des valeurs des bits du compteur, et cette nouvelle valeur est prise en compte par le signal *end_cycle* relié à l'entrée *CE* du registre.

À noter que si les bits 0, 1 et 3 utilisent l'ensemble des bits du compteur pour calculer la prochaine valeur dans des LUT4, le bit 2 n'est pas réinitialisé quand le compteur atteint 10 ('1010') et ne devrait pas être modifié à ce moment si il y avait incrémentation : il n'a pas besoin de connaître l'état du bit 3, et une simple LUT3 est suffisante.

Au total, ce compteur utilise 4 FDCE, 1 LUT3 et 3 LUT4.

- L'état de la FSM est stocké sur 3 bits de registre (2 FDCE et 1 FDPE pour le reset qui remet à l'état rouge '001'), avec une mise-à-jour de chaque bit par une LUT6 qui prend en entrée l'état actuel du bit, celui du bit « précédent » et les 4 bits du compteur de cycle (équivalent du calcul du signal *update_color*).

Au total, la FSM utilise 2 FDCE, 1 FDPE et 3 LUT6.

- Le signal *update_driverA* est créé par une LUT4 qui compare les 4 bits du compteur de cycle (équivalent du calcul du signal *update_color*) et le stocke dans un registre FDCE pour le décaler d'un coup d'horloge.
- Le compteur « stretched » possède 4 bits stockés dans des registres FDCE, la décrémentation/réinitialisation/stagnation de chacun des bits est calculée par une LUT5 qui compare les 4 bits du compteur et le signal *update_driver* (en sortie de la LUT *update_driverA_i_1*).

Au total, ce compteur utilise 4 FDCE et 4 LUT5.

- Le fonctionnement de *LED_driver_bis_0* est presque inchangé par rapport à la partie 2 du TP4, avec :
 - 2 LUT4, 1 FDPE et 1 FDCE pour le code couleur interne *c_code* ;
 - 1 FDCE pour l'état de la FSM *Blink* (on/off) ;
 - 3 LUT3 pour gérer les 3 canaux de LED ;

- Le passage d'une *cible* à « 100 000 000 » change légèrement l'organisation du groupe *Counter_unit*, réduisant légèrement les redondances de recalcul d'équivalent à *end_counter* : seul le bit 15 est maintenant concerné.

Au total, 27 FDCE, 7 CARRY4, 1 LUT1, 16 LUT4, 5 LUT5 et 2 LUT6 sont utilisés.

- Le fonctionnement de *LED_driver_bis_1* est presque identique, avec :
 - 1 LUT4 pour calculer le signal *update_driverB* à partir des 4 bits du compteur « stretched » ;
 - 2 LUT4, 1 FDPE et 1 FDCE pour le code couleur interne *c_code* ;
 - 1 FDCE pour l'état de la FSM *Blink* (on/off) ;
 - 3 LUT3 pour gérer les 3 canaux de LED ;
 - *Counter_unit* est encore une fois un peu modifié, une redondance de calcul d'équivalent à *end_counter* est maintenant présente sur le bit 20.

À noter que le signal *end_cycle* n'est pas construit, puisqu'il est inutilisé ailleurs.

Au total, 27 FDCE, 7 CARRY4, 1 LUT1, 16 LUT4, 4 LUT5 et 2 LUT6 sont utilisés.

On retrouve bien au total les nombres de primitives donnés dans le rapport pour les ressources utilisées (dont 71 LUT et 72 registres). En plus de cela, le rapport d'utilisation prévoit des « Slices » de 64 LUT et 72 FF (registres).

Pour la seconde synthèse (avec *Module_2LED_2clk_fifo*), le rapport de synthèse est dans le fichier *Module_ccd_fifo.vds*. Il nous donne les ressources utilisées suivantes :

```
Report BlackBoxes:
+-----+-----+
|      |BlackBox name      |Instances |
+-----+-----+
|1|    |clk_wiz_0          |         1|
|2|    |fifo_generator_0   |         1|
+-----+-----+

Report Cell Usage:
+-----+-----+
|      |Cell                  |Count |
+-----+-----+
|1|    |clk_wiz_0_bbox      |      1|
|2|    |fifo_generator_0_bbox|      1|
|3|    |CARRY4              |     14|
|4|    |LUT1                 |      2|
|5|    |LUT2                 |      1|
|6|    |LUT3                 |      7|
|7|    |LUT4                 |     42|
|8|    |LUT5                 |      9|
|9|    |LUT6                 |      7|
|10|   |FDCE                 |     65|
|11|   |FDPE                 |      3|
|12|   |IBUF                 |      1|
|13|   |OBUF                 |      6|
+-----+-----+
```

La schématique détaillée de cette synthèse est disponible dans le fichier *Schematic_TP5_stretched.pdf*.

Parmi les nouvelles fonctionnalités mises en places pendant ce TP, on pourra remarquer :

- Il faut 1 IBUF pour l'entrée *resetn* et 6 OBUF pour les 6 canaux de sorties LED.
- Le calcul du *resetn_ccd* est encore effectué dans le sous-groupe *Counter_unit* de *LED_driver_bis_0*, par une LUT2 qui effectue le OR entre *resetn* et le signal *locked* de la PPL.
- Le compteur de cycles à le même fonctionnement. Au total, ce compteur utilise 4 FDCE, 1 LUT3 et 3 LUT4.
- La FSM *Color* a le même fonctionnement. Au total, la FSM utilise 2 FDCE, 1 FDPE et 3 LUT6.
- Le signal *update_driverA* est encore crée par une LUT4 qui compare les 4 bits du compteur de cycle (équivalent du calcul du signal *update_color*) et le stocke dans un registre FDCE pour le décaler d'un coup d'horloge.
- La FIFO possède en entrée :
 - En donnée d'écriture, le signal *update_color* décrit à l'étape précédente.
 - En déclencheur d'écriture la sortie d'une LUT6 qui compare le signal de sortie de la FIFO *full*, le signal *update_driverA* et reconstitue *update_color* sur les 4 bits du compteur de cycles.
 - En déclencheur de lecture la sortie d'une LUT1 qui inverse le signal de sortie de la FIFO *empty*

Au total, les signaux d'entrée de la FIFO utilisent 1 LUT1 et 1 LUT6 spécifiques.

- Le fonctionnement de *LED_driver_bis_0* est identique à la synthèse précédente. Au total, 29 FDCE, 1FDPE, 7 CARRY4, 1 LUT1, 3 LUT3, 18 LUT4, 5 LUT5 et 2 LUT6 sont utilisés.
- Le fonctionnement de *LED_driver_bis_1* est presque identique, avec :
 - 1 LUT4 pour calculer le signal *update_driverB* à partir des 4 bits du compteur « stretched » ;
 - 2 LUT4, 1 FDPE et 1 FDCE pour le code couleur interne *c_code* ;
 - 1 FDCE pour l'état de la FSM *Blink* (on/off) ;
 - 3 LUT3 pour gérer les 3 canaux de LED ;
 - *Counter_unit* est encore une fois un peu modifié, une redondance de calcul d'équivalent à *end_counter* est maintenant présente sur le bit 25, et une LUT4 plutôt que LUT1 est utilisé pour le LSD.

À noter que le signal *end_cycle* n'est pas construit, puisqu'il est inutilisé ailleurs.

Au total, 27 FDCE, 7 CARY4, 17 LUT4, 4 LUT5 et 1 LUT6 sont utilisés.

On retrouve bien au total les nombres de primitives donnés dans le rapport pour les ressources utilisées (dont 68 LUT et 68 registres). En plus de cela, le rapport d'utilisation prévoit des « Slices » de 62 LUT et 68 FF.

Pour le debug sur notre carte, on voudrait en théorie placer de manière générale des sondes ILA sur les signaux suivants :

- Nos six sorties qui pilotent les LEDs (*led_0_r*, *led_0_g* et *led_0_b* pour la LED 0, *led_1_r*, *led_1_g* et *led_1_b* pour la LED 1).
- Le signal sur 4 bits *ctr_cycle* du compteur de cycles.
- Le signal *end_cycle*, reconstitué à partir du *current_state* (FSM *Blink*) et *end_counter* dans *LED_driver_bis_0*.
- Le signal *update_color* qui déclenche le changement d'état de la FSM *Color*
- Le signal sur 3 bits *current_state* qui contient l'état de la FSM *Color*.
- Le signal *update_driverA* qui doit lancer l'*update* du module *LED_driver_bis_0*.
- Les signaux *c_code* interne aux modules *LED_driver_bis*, qui montrent si le signal *update* du module a bien été pris en compte et calculent le code couleur qui détermine la couleur de clignotement de la LED (en fonction de l'état de la FSM).
- Le signal *current_state* de la FSM *Blink* dans *LED_driver_bis_1* qui permettra de vérifier le fonctionnement du clignotement de la LED 1.

En pratique, un problème de Vivado nous empêche de lire les bits 1 et 2 de *current_state* de la FSM *Color*.

Pour la synthèse avec *Module_2LED_2clk_stretched*, on placera en plus des sondes ILA sur les signaux *ctr_stretched* (qui permettent de déduire *update_driverB* quand le compteur est différent de 0).

Pour la synthèse avec *Module_2LED_2clk_fifo*, on placera en plus des sondes ILA sur les signaux spécifiques de la FIFO : *data_out_fifo* (équivalent à *update_driverB*), *write_enable*, *read_enable*, *full* et *empty*.

12) Pour avoir nos rapports de timing, on effectue dans un premier temps l'implémentation sans sondes ILA de nos deux solutions.

De manière générale dans tous nos rapports de timing, on verra un récapitulatif d'horloges montrant qu'on a bien créé les horloges attendues à 50 MHz et 250 MHz :

Clock Summary			

Clock	Waveform(ns)	Period(ns)	Frequency(MHz)
-----	-----	-----	-----
clk	{0.000 4.000}	8.000	125.000
clk_out1_clk_wiz_0	{0.000 2.000}	4.000	250.000
clk_out2_clk_wiz_0	{0.000 10.000}	20.000	50.000
clkfbout_clk_wiz_0	{0.000 4.000}	8.000	125.000
sys_clk_pin	{0.000 4.000}	8.000	125.000
clk_out1_clk_wiz_0_1	{0.000 2.000}	4.000	250.000
clk_out2_clk_wiz_0_1	{0.000 10.000}	20.000	50.000
clkfbout_clk_wiz_0_1	{0.000 4.000}	8.000	125.000

Pour notre première implémentation (avec *Module_2LED_2clk_stretched*), on obtient le rapport de timing du fichier *Module_ccd_timing_summary_routed_stetched.rpt*.

Le récapitulatif de timing obtenu est le suivant :

Design Timing Summary											

WNS(ns)	TNS(ns)	TNS Failing Endpoints	TNS Total Endpoints	WHS(ns)	THS(ns)	THS Failing Endpoints	THS Total Endpoints	WPWS(ns)	TPWS(ns)	TPWS Failing Endpoints	TPWS Total Endpoints
0.072	0.000	0	76	0.123	0.000	0	76	1.500	0.000	0	80

On observe notamment que :

- la valeur de *Worst Negative Slack (WNS)* vaut 0,072 ns, ce qui est positif et indique qu'**il n'y a pas de violation de set up**.
- La valeur de *Worst Hold Slack (WHS)* vaut 0,123 ns, ce qui est positif et indique qu'**il n'y a pas de violation de hold**.

Cependant, bien que positive, la valeur de WNS est très proche de 0. À 250 MHz elle vaut environ 2 % d'une période, ce qui est loin de la marge qu'on recommande de garder.

On a plus de détail sur le chemin critique dans le rapport :

```

-----
From Clock:  clk_out1_clk_wiz_0
To Clock:    clk_out1_clk_wiz_0

Setup :      0 Failing Endpoints, Worst Slack    0.072ns, Total Violation    0.000ns
Hold  :      0 Failing Endpoints, Worst Slack    0.186ns, Total Violation    0.000ns
PW   :      0 Failing Endpoints, Worst Slack    1.500ns, Total Violation    0.000ns
-----

```

Max Delay Paths

```

-----
Slack (MET) :      0.072ns (required time - arrival time)
Source:      Module_2LED_2clk_stretched_0/LED_driver_bis_0/counter_unit_1/Reg_ctr_reg[11]/C
              (rising edge-triggered cell FDCE clocked by clk_out1_clk_wiz_0 {rise@0.000ns fall@2.000ns period=4.000ns})
Destination: Module_2LED_2clk_stretched_0/LED_driver_bis_0/counter_unit_1/Reg_ctr_reg[25]/D
              (rising edge-triggered cell FDCE clocked by clk_out1_clk_wiz_0 {rise@0.000ns fall@2.000ns period=4.000ns})
Path Group:  clk_out1_clk_wiz_0
Path Type:   Setup (Max at Slow Process Corner)
Requirement: 4.000ns (clk_out1_clk_wiz_0 rise@4.000ns - clk_out1_clk_wiz_0 rise@0.000ns)
Data Path Delay: 3.899ns (logic 2.258ns (57.911%) route 1.641ns (42.089%))
Logic Levels:  10 (CARRY4=7 LUT4=1 LUT5=1 LUT6=1)
Clock Path Skew: -0.028ns (DCD - SCD + CPR)
  Destination Clock Delay (DCD): -1.829ns = ( 2.171 - 4.000 )
  Source Clock Delay (SCD): -2.166ns
  Clock Pessimism Removal (CPR): -0.365ns
Clock Uncertainty: 0.063ns ((TSJ^2 + DJ^2)^1/2) / 2 + PE
  Total System Jitter (TSJ): 0.071ns
  Discrete Jitter (DJ): 0.105ns
  Phase Error (PE): 0.000ns
-----

```

On confirme que ce chemin à lieu sur la sortie *clk_out_1* de la PLL, donc notre horloge à 250 MHz.

Plus précisément, il a lieu dans la partie *Counter_unit* du module *LED_driver_bis_0*. Ce chemin passe par 7 CARRY4 et 3 LUT différentes.

En regardant le détail du chemin, on se rend compte qu'il part du registre du bit 11 du compteur, passe dans 3 LUT pour calculer l'équivalence à *end_counter* et être combiné au LSD pour savoir si il faut déclencher une incrémentation (entrée *DI[0]* du CARRY4 des 4 premiers bits) avant de se propager à chacun des 6 autres CARRY4 en sortant par *CO[3]* et rentrant par *CI*, jusqu'à ressortir par la sortie *O[1]* du dernier CARRY4 et être stocké dans le registre du bit 25.

Pour notre deuxième implémentation (avec *Module_2LED_2clk_fifo*), on obtient le rapport de timing du fichier *Module_ccd_timing_summary_routed_fifo.rpt*.

Le récapitulatif de timing obtenu est le suivant :

Design Timing Summary											
WNS(ns)	TNS(ns)	TNS Failing Endpoints	TNS Total Endpoints	WHS(ns)	THS(ns)	THS Failing Endpoints	THS Total Endpoints	WPWS(ns)	TPWS(ns)	TPWS Failing Endpoints	TPWS Total Endpoints
0.025	0.000	0	79	0.042	0.000	0	79	1.424	0.000	0	89

On observe notamment que :

- la valeur de *Worst Negative Slack (WNS)* vaut 0,025 ns, ce qui est positif et indique qu'il n'y a pas de violation de set up.
- La valeur de *Worst Hold Slack (WHS)* vaut 0,042 ns, ce qui est positif et indique qu'il n'y a pas de violation de hold.

Cependant, bien que positive, la valeur de WNS est très proche de 0. À 250 MHz elle vaut environ 0,6 % d'une période, ce qui est loin de la marge qu'on recommande de garder.

On a plus de détail sur le chemin critique dans le rapport :

```
-----
From Clock:  clk_out1_clk_wiz_0
To Clock:    clk_out1_clk_wiz_0

Setup :      0 Failing Endpoints, Worst Slack      0.025ns, Total Violation      0.000ns
Hold  :      0 Failing Endpoints, Worst Slack      0.190ns, Total Violation      0.000ns
PW   :      0 Failing Endpoints, Worst Slack      1.424ns, Total Violation      0.000ns
-----

Max Delay Paths
-----
Slack (MET) :      0.025ns (required time - arrival time)
Source:      Module_2LED_2clk_fifo_0/LED_driver_bis_0/counter_unit_1/Reg_ctr_reg[16]/C
              (rising edge-triggered cell FDCE clocked by clk_out1_clk_wiz_0 {rise@0.000ns fall@2.000ns period=4.000ns})
Destination: Module_2LED_2clk_fifo_0/LED_driver_bis_0/counter_unit_1/Reg_ctr_reg[25]/D
              (rising edge-triggered cell FDCE clocked by clk_out1_clk_wiz_0 {rise@0.000ns fall@2.000ns period=4.000ns})
Path Group:   clk_out1_clk_wiz_0
Path Type:    Setup (Max at Slow Process Corner)
Requirement:  4.000ns (clk_out1_clk_wiz_0 rise@4.000ns - clk_out1_clk_wiz_0 rise@0.000ns)
Data Path Delay: 3.948ns (logic 2.134ns (54.059%) route 1.814ns (45.941%))
Logic Levels:   9 (CARRY4=7 LUT4=1 LUT6=1)
Clock Path Skew: -0.026ns (DCD - SCD + CPR)
  Destination Clock Delay (DCD): -1.899ns = ( 2.101 - 4.000 )
  Source Clock Delay (SCD): -2.242ns
  Clock Pessimism Removal (CPR): -0.369ns
Clock Uncertainty: 0.063ns ((TSJ^2 + DJ^2)^1/2) / 2 + PE
  Total System Jitter (TSJ): 0.071ns
  Discrete Jitter (DJ): 0.105ns
  Phase Error (PE): 0.000ns
```

On confirme que ce chemin à lieu sur la sortie *clk_out_1* de la PLL, donc notre horloge à 250 MHz.

Plus précisément, il a lieu dans la partie *Counter_unit* du module *LED_driver_bis_0*. Ce chemin passe par 7 CARRY4 et 2 LUT différentes.

En regardant le détail du chemin, on se rend compte qu'il part du registre du bit 16 du compteur, passe dans 2 LUT pour calculer l'équivalence à *end_counter* et être combiné au LSD pour savoir si il faut déclencher une incrémentation (entrée *DI[0]* du CARRY4 des 4 premiers bits) avant de se propager à chacun des 6 autres CARRY4 en sortant par *CO[3]* et rentrant par *CI*, jusqu'à ressortir par la sortie *O[1]* du dernier CARRY4 et être stocké dans le registre du bit 25.

Dans nos deux implémentations, notre chemin critique est créé par notre module *Counter_unit* : à 250 MHz, effectuer la comparaison à la cible (via 2 ou 3 LUT) pour savoir si il faut réinitialiser notre compteur ou incrémenter, puis propager l'éventuel incrémentation à travers 7 CARRY4 est trop long.

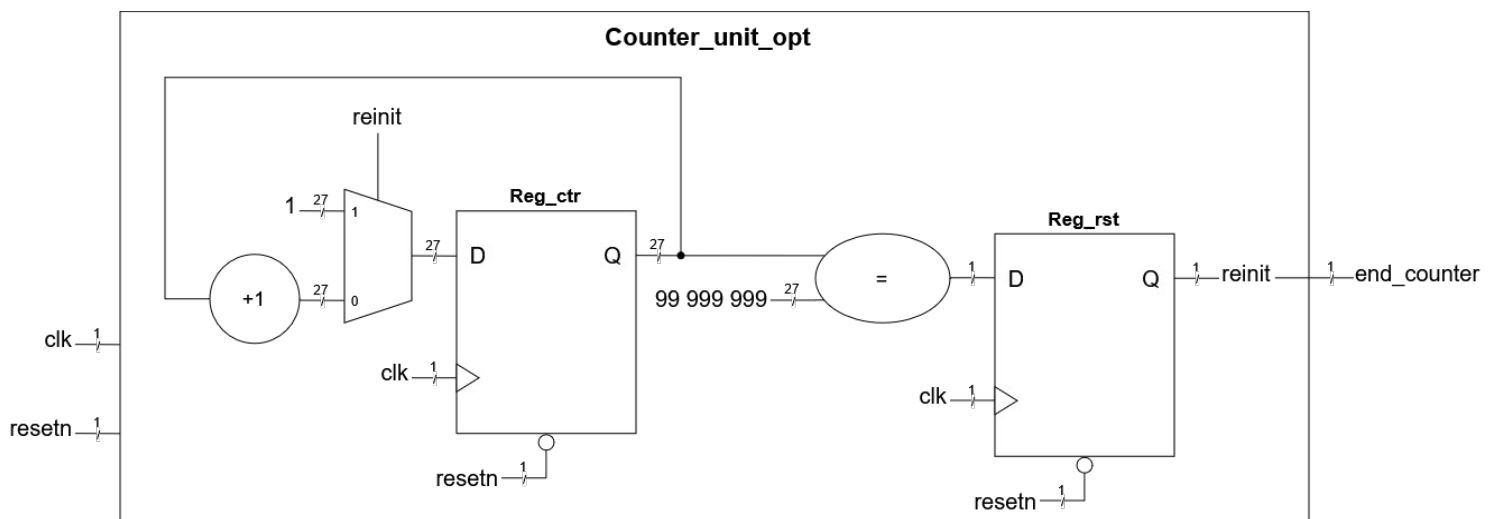
J'ai donc décidé d'optimiser le design de mon module *Counter_unit* pour que ces deux opérations ne soient plus enchaînées entre 2 registres, en implémentant une étape intermédiaire.

Pour cela j'enregistre le résultat de ma comparaison à la cible dans un registre, qui produit le signal *reinit* indiquant la réinitialisation de mon compteur. Ce signal interne *reinit* est aussi relié à la sortie du module, *end_counter*.

Puisqu'il passe par un registre, le signal *reinit* est décalé d'un coup d'horloge par rapport à notre comparaison. Pour que la réinitialisation se déclenche bien au moment où la cible est atteinte, on fait la comparaison avec une valeur « cible - 1 ».

Cette comparaison avec « cible - 1 » nous évite aussi d'avoir à modifier le calcul de la taille (en bits) de notre compteur, comme il m'a été suggéré en correction du TP3, la taille calculée par ma formule étant maintenant suffisante pour contenir toutes les informations utiles du compteur.

Voici le schéma de la nouvelle architecture RTL du module, *Counter_unit_opt*, pour une valeur de cible=10 000 000 :



L'implémentation en VHDL de cette architecture est faite dans le fichier *counter_unit_opt.vhd*.

En plus de l'ajout du registre produisant le signal *Reg_rst* qui remplace l'ancien signal *r_cible*, et la modification de la comparaison du compteur pour être maintenant faite avec « cible - 1 », on a fait quelques optimisations à notre implémentation VHDL :

- On calcule la valeur suivante du compteur en deux étapes, d'abord réinitialisation à 0 si cible atteinte, puis incrémentation dans tous les cas (au lieu de faire soit incrémentation, soit réinitialisation à 1).
- On précise que la réinitialisation ne se fait que pour les bits à '1' de la cible, et non l'ensemble des bits (grâce à une boucle for).

Pour la suite du TP, on remplace l'ancien module *Counter_unit* par le nouveau *Counter_unit_opt* dans le code VHDL de *LED_driver_bis.vhd*.

On refait deux synthèses avec cette optimisation.

Pour la synthèse utilisant *Module_2LED_2clk_stretched*, on enregistre le rapport de synthèse dans le fichier *Module_ccd_stretched_opt.vds*. Il nous donne les ressources utilisées suivantes :

```

Report BlackBoxes:
+-----+-----+-----+
|      |BlackBox name |Instances |
+-----+-----+-----+
|1      |clk_wiz_0      |1|
+-----+-----+-----+

Report Cell Usage:
+-----+-----+-----+
|      |Cell           |Count |
+-----+-----+-----+
|1      |clk_wiz_0_bbox |1|
|2      |CARRY4         |14|
|3      |LUT1           |2|
|4      |LUT2           |28|
|5      |LUT3           |9|
|6      |LUT4           |7|
|7      |LUT5           |6|
|8      |LUT6           |14|
|9      |FDCE           |71|
|10     |FDPE           |3|
|11     |IBUF           |1|
|12     |OBUF           |6|
+-----+-----+-----+

```

La schématique de cette synthèse après optimisation de *Counter_unit* est disponible dans *Schématique_TP5_stretched_opt.pdf*. Voici ce qu'on observe :

- Il faut toujours 1 IBUF pour l'entrée *resetrn* et 6 OBUF pour les 6 canaux de sorties LED.
- *resetrn_ccd* est maintenant calculé dans le sous-groupe *Counter_unit_opt* de *LED_driver_bis_1*, toujours par 1 LUT2.
- Le compteur de cycles est inchangé, il utilise toujours 4 FDCE, 1 LUT3 et 3 LUT4.
- L'état de la FSM *Color* est inchangé, utilisant au total 2 FDCE, 1 FDPE et 3 LUT6.
- Le signal *update_driverA* est toujours créé par 1 LUT4 et 1 FDCE.
- Le compteur « stretched » est pratiquement inchangé, utilisant toujours 4 FDCE et 4 LUT5 , et 1 LUT4 supplémentaire qui recalcule *update_color*.
- En dehors de la partie *Counter_unit*, le fonctionnement des *LED_driver_bis_0* et *LED_driver_bis_1* sont inchangés, avec au total 4 FDCE, 2 FDPE, 1 LUT2, 6 LUT3, 2 LUT4 et 2 LUT6.
- Les groupe *Counter_unit_opt* sont profondément changés, avec pour celui dans *LED_driver_bis_0* :
 - Les 27 bits du compteur sont stockés sur autant de registres FDCE.
 - La comparaison à la cible (- 1) est effectuée sur l'ensemble des bits du compteur par un total de 6 LUT *Reg_rst_i_1* à *Reg_rst_i_6* (1 LUT3 et 5 LUT6) puis stocké dans un registre FDCE *Reg_rst_reg*.
 - Le LSD du compteur alterne à chaque coup d'horloge entre '0' et '1' grâce à une LUT1 *Reg_ctr[0]_i_1*.
 - La valeur avant inversion du LSD du compteur sert d'entrée d'incrémentation *CYINIT* à un CARRY4 qui l'applique sur les bits 1 à 4. Par tranche de 4, les bits suivants sont si

nécessaire réinitialisé par une LUT2 (prenant en entrée la valeur du bit et le signal de reset en sortie de son registre) (cette LUT2 n'existe que si le bit vaut '1' dans la cible), puis transmis à un CARRY4 pour se voir propager l'incrémentation en retenue du CARRY4 précédent.

Cela fait un total de 7 CARRY4 et 12 LUT2.

○ Dans ce groupe sont aussi calculés :

- l'état de la FSM *Blink* du module *LED_Driver_bis_0*, avec une LUT2 prenant en entrée l'ancien état de la FSM et le signal du registre *Reg_rst* (équivalent de *end_counter*)
- le signal *end_cycle*, avec une LUT6 prenant en entrée le signal du registre *Reg_rst_reg*, l'état de la FSM *Blink* et les 4 bits du compteur de cycles ;

Au total, 28 FDCE, 7 CARRY4, 1 LUT1, 13 LUT2, 1 LUT3, 1 LUT5 et 5 LUT6 sont utilisés dans ce groupe.

- Pour le groupe *Counter_unit_opt* de *LED_driver_bis_1*, le fonctionnement est identique, à l'exception du calcul de *end_cycle*.

Au total, 28 FDCE, 7 CARRY4, 1 LUT1, 13 LUT2, 1 LUT3, 1 LUT5 et 4 LUT6 sont utilisés dans ce groupe.

On retrouve bien au total les nombres de primitives donnés dans le rapport pour les ressources utilisées (dont 66 LUT et 74 registres). En plus de cela, le rapport d'utilisation prévoit des « Slices » de 58 LUT et 74 FF (registres).

Pour la synthèse utilisant *Module_2LED_2clk_fifo*, on enregistre le rapport de synthèse dans le fichier *Module_ccd_fifo_opt.vds*. Il nous donne les ressources utilisées suivantes :

```
Report BlackBoxes:
+-----+-----+-----+
|      |BlackBox name      |Instances |
+-----+-----+-----+
|1      |clk_wiz_0           |          |1|
|2      |fifo_generator_0    |          |1|
+-----+-----+-----+

Report Cell Usage:
+-----+-----+-----+
|      |Cell                |Count  |
+-----+-----+-----+
|1      |clk_wiz_0_bbox       |        |1|
|2      |fifo_generator_0_bbox|        |1|
|3      |CARRY4               |       |14|
|4      |LUT1                 |        |3|
|5      |LUT2                 |       |27|
|6      |LUT3                 |        |9|
|7      |LUT4                 |        |8|
|8      |LUT5                 |        |2|
|9      |LUT6                 |       |13|
|10     |FDCE                 |       |67|
|11     |FDPE                 |        |3|
|12     |IBUF                 |        |1|
|13     |OBUF                 |        |6|
+-----+-----+-----+
```

La schématique de cette synthèse après optimisation de *Counter_unit* est disponible dans *Schematic_TP5_stretched_opt.pdf*. Voici ce qu'on observe :

- Il faut toujours 1 IBUF pour l'entrée *resetn* et 6 OBUF pour les 6 canaux de sorties LED.
- *resetn_ccd* est maintenant lui aussi calculé dans le sous-groupe *Counter_unit_opt* de *LED_driver_bis_1*, toujours par 1 LUT2.
- Le compteur de cycles est inchangé, il utilise toujours 4 FDCE, 1 LUT3 et 3 LUT4.
- L'état de la FSM *Color* est inchangé, utilisant au total 2 FDCE, 1 FDPE et 3 LUT6.
- Le signal *update_driverA* est toujours créé par 1 LUT4 et 1 FDCE.
- La FIFO utilise toujours 1 LUT1 et 1 LUT6 spécifiques.
- En dehors de la partie *Counter_unit*, le fonctionnement des *LED_driver_bis_0* et *LED_driver_bis_1* sont inchangés, avec au total 4 FDCE, 2 FDPE, 6 LUT3 et 4 LUT4.
- Les groupe *Counter_unit_opt* sont identiques à la synthèse précédente :
 - Dans le groupe *LED_driver_bis_0*, *Counter_unit_opt* utilise au total 28 FDCE, 7 CARRY4, 1 LUT1, 13 LUT2, 1 LUT3, 1 LUT5 et 5 LUT6.
 - Dans le groupe *LED_driver_bis_1*, *Counter_unit_opt* utilise au total 28 FDCE, 7 CARRY4, 1 LUT1, 13 LUT2, 1 LUT3, 1 LUT5 et 4 LUT6.

On retrouve bien au total les nombres de primitives donnés dans le rapport pour les ressources utilisées (dont 62 LUT et 70 registres). En plus de cela, le rapport d'utilisation prévoit des « Slices » de 56 LUT et 70 FF (registres).

On effectue ensuite l'implémentation de ces deux variantes optimisées.

Pour notre première implémentation (avec *Module_2LED_2clk_stretched*), on obtient le rapport de timing du fichier *Module_ccd_timing_summary_routed_stetched_opt.rpt*.

Le récapitulatif de timing obtenu est le suivant :

Design Timing Summary											
WNS(ns)	TNS(ns)	TNS Failing Endpoints	TNS Total Endpoints	WHS(ns)	THS(ns)	THS Failing Endpoints	THS Total Endpoints	WPWS(ns)	TPWS(ns)	TPWS Failing Endpoints	TPWS Total Endpoints
0.555	0.000	0	78	0.150	0.000	0	78	1.500	0.000	0	82

On observe notamment que :

- la valeur de *Worst Negative Slack (WNS)* vaut 0,555 ns, ce qui est positif et indique qu'**il n'y a pas de violation de set up**.
- La valeur de *Worst Hold Slack (WHS)* vaut 0,150 ns, ce qui est positif et indique qu'**il n'y a pas de violation de hold**.

On a plus de détail sur le chemin critique dans le rapport :


```

-----
From Clock:  clk_out1_clk_wiz_0
To Clock:    clk_out1_clk_wiz_0

Setup :      0 Failing Endpoints, Worst Slack      0.555ns, Total Violation      0.000ns
Hold  :      0 Failing Endpoints, Worst Slack      0.231ns, Total Violation      0.000ns
PW   :      0 Failing Endpoints, Worst Slack      1.500ns, Total Violation      0.000ns
-----

```

Max Delay Paths

```

-----
Slack (MET) :      0.555ns (required time - arrival time)
Source:      Module_2LED_2clk_stretched_0/LED_driver_bis_0/counter_unit_1/Reg_ctr_reg[1]/C
              (rising edge-triggered cell FDCE clocked by clk_out1_clk_wiz_0 {rise@0.000ns fall@2.000ns period=4.000ns})
Destination: Module_2LED_2clk_stretched_0/LED_driver_bis_0/counter_unit_1/Reg_ctr_reg[26]/D
              (rising edge-triggered cell FDCE clocked by clk_out1_clk_wiz_0 {rise@0.000ns fall@2.000ns period=4.000ns})
Path Group:  clk_out1_clk_wiz_0
Path Type:   Setup (Max at Slow Process Corner)
Requirement: 4.000ns (clk_out1_clk_wiz_0 rise@4.000ns - clk_out1_clk_wiz_0 rise@0.000ns)
Data Path Delay: 3.414ns (logic 2.016ns (59.047%) route 1.398ns (40.953%))
Logic Levels:  7 (CARRY4=7)
Clock Path Skew: -0.030ns (DCD - SCD + CPR)
  Destination Clock Delay (DCD): -1.829ns = ( 2.171 - 4.000 )
  Source Clock Delay (SCD): -2.164ns
  Clock Pessimism Removal (CPR): -0.365ns
Clock Uncertainty: 0.063ns ((TSJ^2 + DJ^2)^1/2) / 2 + PE
  Total System Jitter (TSJ): 0.071ns
  Discrete Jitter (DJ): 0.105ns
  Phase Error (PE): 0.000ns
-----

```

On confirme que ce chemin à lieu sur la sortie *clk_out_1* de la PLL, donc notre horloge à 250 Mhz, toujours dans la partie *Counter_unit* du module *LED_driver_bis_0*.

Ce chemin critique à un slack de 0,555 ns, soit 13,9 % d'une période de *clkA*. Si on atteint toujours pas le pourcentage recommandé de 20 % de la période, on a quand même multiplié notre WNS par plus de 7 par rapport à l'implémentation sans optimisation de *Counter_unit*.

On remarquera que ce chemin critique relie les bits 1 à 26 du compteur en passant par les 7 CARRY4.

Une bonne partie de notre délais de propagation est lié au parcours entre le registre du bit 1 et le premier CARRY4. Une possibilité pour améliorer notre slack pourrait être de raccourcir ce délais avec des contraintes de placement à l'implémentation : en regardant le détail d'implémentation, on se rend compte que pour la slice SLICE_X51Y61 où est implémenté ce fil, son délais vaut environ 1,500 ns, alors que dans d'autres cellules le délais entre un registre et l'entrée S du CARRY4 de la même slice vaut plutôt autour de 0,500 ns.

Pour notre deuxième implémentation (avec *Module_2LED_2clk_fifo*), on obtient le rapport de timing du fichier *Module_ccd_timing_summary_routed_fifo_opt.rpt*.

Le récapitulatif de timing obtenu est le suivant :

Design Timing Summary											
WNS(ns)	TNS(ns)	TNS Failing Endpoints	TNS Total Endpoints	WHS(ns)	THS(ns)	THS Failing Endpoints	THS Total Endpoints	WPWS(ns)	TPWS(ns)	TPWS Failing Endpoints	TPWS Total Endpoints
0.524	0.000	0	81	0.080	0.000	0	81	1.424	0.000	0	91

On observe notamment que :

- la valeur de *Worst Negative Slack (WNS)* vaut 0,524 ns, ce qui est positif et indique qu'**il n'y a pas de violation de set up**.
- La valeur de *Worst Hold Slack (WHS)* vaut 0,080 ns, ce qui est positif et indique qu'**il n'y a pas de violation de hold**.

On a plus de détail sur le chemin critique dans le rapport :

```
-----
From Clock:  clk_outl_clk_wiz_0
To Clock:    clk_outl_clk_wiz_0

Setup :      0 Failing Endpoints, Worst Slack      0.524ns, Total Violation      0.000ns
Hold  :      0 Failing Endpoints, Worst Slack      0.199ns, Total Violation      0.000ns
PW   :      0 Failing Endpoints, Worst Slack      1.424ns, Total Violation      0.000ns
-----

Max Delay Paths
-----
Slack (MET) :      0.524ns (required time - arrival time)
Source:      Module_2LED_2clk_fifo_0/LED_driver_bis_0/counter_unit_1/Reg_ctr_reg[1]/C
              (rising edge-triggered cell FDCE clocked by clk_outl_clk_wiz_0 {rise@0.000ns fall@2.000ns period=4.000ns})
Destination: Module_2LED_2clk_fifo_0/LED_driver_bis_0/counter_unit_1/Reg_ctr_reg[26]/D
              (rising edge-triggered cell FDCE clocked by clk_outl_clk_wiz_0 {rise@0.000ns fall@2.000ns period=4.000ns})
Path Group:   clk_outl_clk_wiz_0
Path Type:    Setup (Max at Slow Process Corner)
Requirement:  4.000ns (clk_outl_clk_wiz_0 rise@4.000ns - clk_outl_clk_wiz_0 rise@0.000ns)
Data Path Delay: 3.494ns (logic 2.063ns (59.039%) route 1.431ns (40.961%))
Logic Levels:  7 (CARRY4=7)
Clock Path Skew: -0.027ns (DCD - SCD + CPR)
  Destination Clock Delay (DCD): -1.899ns = ( 2.101 - 4.000 )
  Source Clock Delay (SCD): -2.241ns
  Clock Pessimism Removal (CPR): -0.369ns
Clock Uncertainty: 0.063ns ((TSJ^2 + DJ^2)^1/2) / 2 + PE
  Total System Jitter (TSJ): 0.071ns
  Discrete Jitter (DJ): 0.105ns
  Phase Error (PE): 0.000ns
```

On confirme que ce chemin à lieu sur la sortie *clk_out_1* de la PLL, donc notre horloge à 250 Mhz, toujours dans la partie *Counter_unit* du module *LED_driver_bis_0*.

Ce chemin critique à un slack de 0,524 ns, soit 13,1 % d'une période de *clkA*. Si on atteint toujours pas le pourcentage recommandé de 20 % de la période, on a quand même multiplié notre WNS par plus de 20 par rapport à l'implémentation sans optimisation de *Counter_unit*.

On remarquera que ce chemin critique relie les bits 1 à 26 du compteur en passant par les 7 CARRY4.

Une bonne partie de notre délais de propagation est lié au parcours entre le registre du bit 1 et le premier CARRY4. Une possibilité pour améliorer notre slack pourrait être de raccourcir ce délais avec des contraintes de placement à l'implémentation : en regardant le détail d'implémentation, on se rend compte que pour la slice SLICE_X34Y54 où est implémenté ce fil, son délais vaut environ 1,500 ns, alors que dans d'autres cellules le délais entre un registre et l'entrée S du CARRY4 de la même slice vaut plutôt autour de 0,500 ns.

Pour notre application sur carte dans des conditions normales, et sans système critique traité par notre utilisation, on se contentera de ces valeurs de WNS à environ 14 % d'une période.

13) On partira de l'implémentation de la variante *Module_2LED_2clk_stretched* en rajoutant les contraintes des sondes ILA pour générer le bitstream et tester le programme sur carte.

La vidéo *Démo_module_ccd.mp4* présente 26 secondes de fonctionnement de la carte programmée.

La vidéo commence alors que la carte est bloqué en resetn (bouton 0 appuyé).

Le bouton est relâché au début de la vidéo, laissant la carte suivre son fonctionnement normal.

De la 1^{ère} à la 8^{ème} seconde, les LEDs clignotent en rouge, 10 fois pour la LED 0 et 2 fois pour la LED 1.

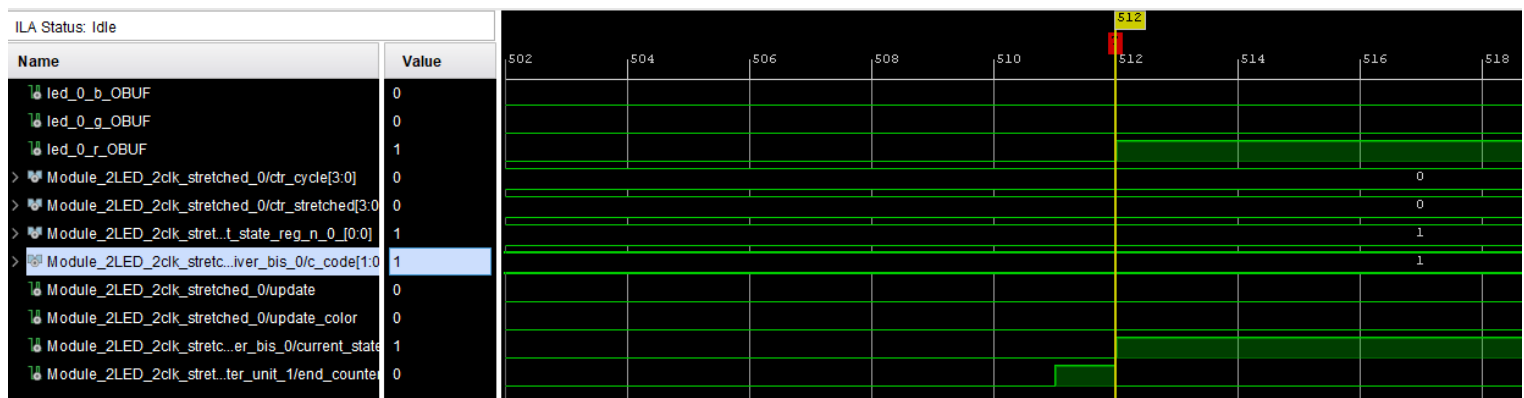
De la 9^{ème} à la 16^{ème} seconde, les LEDs clignotent en bleu, 10 fois pour la LED 0 et 2 fois pour la LED 1.

De la 17^{ème} à la 24^{ème} seconde, les LEDs clignotent en vert, 10 fois pour la LED 0 et 2 fois pour la LED 1.

À partir de la 25^{ème} seconde les LED recommencent à clignoter en rouge, avec 3 clignotements visible pour la LED 0 et un pour la LED 1 avant la fin de la vidéo

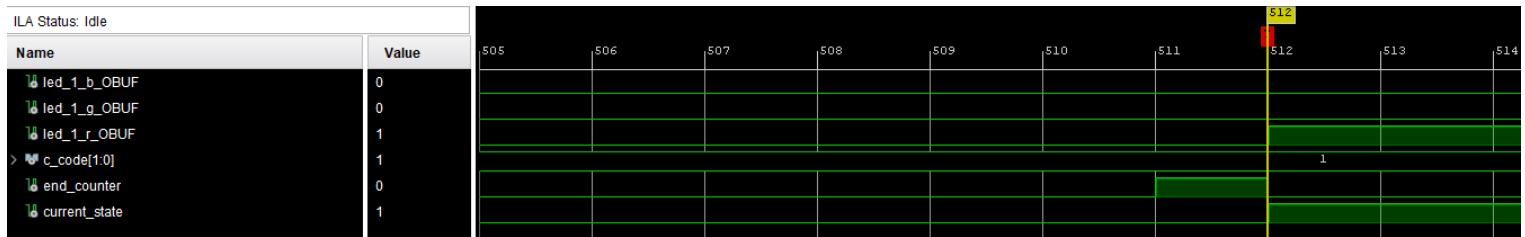
En plus de cette évaluation visuelle, on effectue plusieurs tests avec la ILA.

Le premier teste le fonctionnement de la LED 0, avec un trigger sur un front montant de *led_0_g_OBUF*.



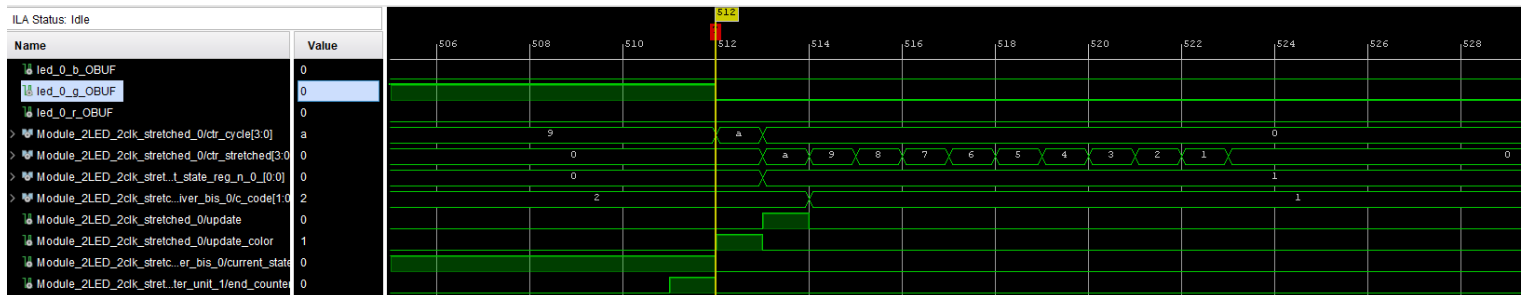
On observe bien que le signal *end_counter* change l'état de la FSM *Blink* de '0' à '1', faisant s'allumer la LED. La couleur d'allumage dépend de la valeur du signal *c_code*, qui vaut dans notre cas '01' ce qui correspond à la couleur rouge.

Le second teste le fonctionnement de la LED 1, avec un trigger sur un front montant de *led_1_g_OBUF*.



On observe bien que le signal *end_counter* change l'état de la FSM *Blink* de '0' à '1', faisant s'allumer la LED. La couleur d'allumage dépend de la valeur du signal *c_code*, qui vaut dans notre cas '01' ce qui correspond à la couleur rouge.

Le troisième teste le changement de couleur après 10 clignotements. Il est trigger par *update_color* qui vaut '1', lancé alors que la LED clignote en vert.



On observe bien que le signal *end_cycle* (*end_counter* et *current_state* de FSM *Blink* à '1') fait passer le compteur de cycle de 9 à 10 ('a' en hexadécimal).

Cela déclenche le signal *update_color* qui va avoir 2 conséquences. La première est de faire changer l'état de la FSM *Color* (visible uniquement sur le bit 0 de l'état, *current_state_reg_n_0* qui passe de '0' à '1'), et de modifier le *color_code* associé (pas de sonde ILA pour ce signal)

La seconde est de déclencher un coup d'horloge plus tard le signal *update_driverA* (simplement nommé *update* sur notre chronogramme). Cela va lancer au coup d'horloge suivant la mise-à-jour de *LED_driver_bis_0*, faisant passer son code couleur interne *c_code* de '0' à '1'. Cela va aussi initialiser à 10 le compteur « stretched » (*ctr_stretched*), qui va ensuite décrémenter à chaque coup d'horloge, jusqu'à stagner à nouveau à 0, 10 coups d'horloge plus tard. Pendant ces 10 coups d'horloge, le signal *update_driverB* (pas possible avec notre synthèse de l'observer à l'ILA) doit être activé, cette durée devant être suffisante pour coïncider avec au moins un front montant de *clkB* et lancer la mise à jour de *LED_driver_bis_1* (le fonctionnement de l'ILA ne permet pas de l'observer en même temps, car elle ne peut acquérir que sur une horloge à la fois)

Au final, que ce soit en visuel ou via l'ILA, la carte semble parfaitement fonctionner.